

LEAVE MANAGEMENT + PAYROLL

Features, Workflows & Integration Architecture

A complete specification of Module 2 (Leave) and Module 3 (Payroll) — every feature, every workflow, and exactly how both connect into the existing attendance system, Module 1, as one continuous employee-cost platform.

Prepared for internal engineering & product planning

Covers: Phase 0 (Foundation) through Phase 4 (Extended Finance Features)

Document scope: architecture, database schema, business workflows, role responsibilities

Table of Contents

Table of Contents	2
1. Executive Summary	5
2. System Overview — Three Modules, One Platform	6
2.1 The dependency chain	6
2.2 What makes this different from a typical student/college EMS	6
3. Module 1 Recap — Attendance (Existing, In Production)	7
3.1 Organization & access model	7
3.2 Attendance verification	7
3.3 Supporting features already in place	7
4. Module 2 — Leave Management: Features	8
4.1 Leave Types & Policies	8
4.2 The Leave Ledger — append-only balance tracking	8
4.3 Accrual	8
4.4 Leave Requests	8
4.5 Comp-Off	8
4.6 Restricted / Floating Holidays	9
4.7 Leave Encashment	9
4.8 Loss of Pay (LOP) — the join to Payroll	9
5. Module 2 — Leave Management: Workflows	11
5.1 Employee workflow	11
5.2 Approver workflow	11
5.3 Automated / system workflow	11
5.4 The professional workflow — how a real HR team actually runs this	11
6. Module 3 — Payroll: Features	13
6.1 Salary Structures & Components	13
6.2 Payroll Component Catalog	13
6.3 Statutory Compliance (India)	13
6.4 The Payroll Run — a controlled processing unit	13
6.5 The Financial Ledger — single source of truth	13
6.6 Loans & Salary Advances	14

6.7 Reimbursements	14
6.8 Cost Centers	14
6.9 Bank Accounts & Payslips	14
6.10 Payroll Validation Report	14
6.11 Payroll Simulation	15
6.12 Multi-Approval, Accounting/Bank Export, Budgets	15
7. Module 3 — Payroll: Workflows	16
7.1 The monthly payroll close — a full-month timeline	16
7.2 Employee-facing workflow	16
7.3 HR / Finance workflow	16
7.4 New joiner → Payroll	17
7.5 Employee exit → Final Settlement	17
7.6 Year-end	17
7.7 Payroll Correction / Retro-Pay	17
8. Cross-Module Integration — The Foundation Layer	19
8.1 Tenant-level timezone & currency	19
8.2 Real policy versioning	19
8.3 Role & privilege extension	19
8.4 Scheduler — organized by cadence	19
8.5 The two ledgers, side by side	19
9. Notification Engine — Systematic, Every Time	21
9.1 One dispatch function, both channels, always together	21
9.2 Schema — extending, not replacing, the existing table	21
9.3 Every event, by recipient — the complete coverage matrix	22
9.4 In-App Notification Center	22
9.5 Migration path for the twelve existing email functions	23
10. Feature Entitlement Cascade	24
10.1 The hierarchy and the one rule that secures it	24
10.2 One catalog — modules and administrative actions, identical treatment	24
10.3 Schema	24
10.4 Onboarding — deny-by-default, not opt-out	25
10.5 Feature dependencies — confirm and cascade, both directions	25

10.6 Migrating off the existing privileges array	26
10.7 The runtime check — one function, everywhere	26
10.8 Every dynamic surface, not just navigation	26
10.9 The full lifecycle, end to end	27
10.10 Deliberately deferred — SaaS commercial features, not EMS features	27
11. Organization Setup & Bulk Operations	28
11.1 “Day 0” — new tenant setup	28
11.2 Bulk Onboarding	28
11.3 Bulk Offboarding & the Exit Checklist	29
12. Roles & Responsibilities Matrix	30
13. Data Model Summary	31
13.1 Foundation (shared by both modules)	31
13.2 Leave Management	31
13.3 Payroll	31
14. Phased Implementation Roadmap	33
15. Design Principles — What Ties This Together	34
Appendix A — Restaurant Vertical Extension (Optional)	35
A.1 Prime Cost / Labor-Cost-to-Sales	35
A.2 Tip-Out Distribution Chains	35
A.3 Meal-Period & Covers-Driven Scheduling	35
A.4 Emergency Shift-Swap Broadcast	35
A.5 Station Certification-Gated Scheduling	35
A.6 Split-Shift Premium Automation	35
A.7 Comp / Void / Walkout ↔ Shift Accountability	35
A.8 Staff Meal as a Tracked Benefit	35

1. Executive Summary

Smart Teams began as a single-purpose attendance system: geofenced, face-verified, audit-logged check-in and check-out for distributed teams. This document specifies the next two modules — **Leave Management** and **Payroll** — designed not as bolt-on features, but as a natural continuation of the same architecture that already runs attendance in production.

The guiding principle across both new modules is a single one: **everything that costs the company money, or affects an employee's time-off balance, becomes an immutable ledger entry**. Every dashboard, every report, and every audit question is answered by querying that ledger — not by writing bespoke calculation logic per screen. This is the same discipline the existing attendance module already applies to its hash-chained audit log; Leave and Payroll simply extend it to time-off balances and money.

This document is organized in three layers: **what exists today** (Module 1, for context), **what is being built** (Modules 2 and 3 — every feature and every workflow, including how a real HR/payroll department would actually run this month to month), and **how it all connects** (the shared ledger, policy versioning, and role-based access control that make the three modules behave as one platform instead of three separate apps).

Layer	Contents
Module 1 — Attendance (existing)	Multi-tenant RBAC, GPS/WiFi/Face/QR verification, breaks, corrections, WFH, holidays, audit ledger
Module 2 — Leave (new)	Leave types, versioned policies, an append-only leave ledger, accrual, approval, comp-off, encashment
Module 3 — Payroll (new)	Salary structures, statutory compliance, payroll runs, the financial ledger, loans, reimbursements, cost centers
Integration layer	One shared ledger philosophy, one policy-versioning mechanism, one RBAC system, one scheduler, one systematic notification engine (in-app + email, every event)

2. System Overview — Three Modules, One Platform

Smart Teams is not three separate applications sharing a login page. It is one tenant-scoped platform where each module produces data the next module consumes — the same way a real company's HR and finance departments depend on each other's output.

2.1 The dependency chain

How data flows through the platform

MODULE 1 – ATTENDANCE

Daily check-in/check-out, GPS + WiFi + Face + QR verification, breaks, corrections

|

v (attendance % , unexcused absences, holiday/weekend work)

MODULE 2 – LEAVE MANAGEMENT

Leave balance ledger, requests, approvals, comp-off, encashment

|

v (Loss-of-Pay days, encashable balance, comp-off credits)

MODULE 3 – PAYROLL

Salary structures, statutory deductions, payroll runs, loans, reimbursements

|

v (every earning / deduction / employer cost, as a row)

THE FINANCIAL LEDGER

One immutable, append-only table – the single source of truth for money

|

v

ANALYTICS & DASHBOARDS

Drill-down cost reports: company -> department -> employee -> transaction

Nothing above is a separate database or a separate service. It is one Postgres schema, one Express backend, one React frontend — the same monorepo that already runs attendance. Leave and Payroll are new tables and new routes inside that same system, reusing its authentication, its role/privilege model, its background scheduler, and its notification/email pipeline.

2.2 What makes this different from a typical student/college EMS

- **Nothing is ever silently overwritten.** Leave balances, salary changes, and policy rates are all versioned or ledgered — history is always reconstructable exactly as it was at the time.
- **Every policy has an effective date.** A PF-rate change in October never alters what March's payslip should have paid.
- **Every dashboard number is explainable.** “Why is this employee's PF ■1,800?” has a literal answer stored alongside the number, not just a total.
- **Attendance, Leave, and Payroll speak the same language.** A Loss-of-Pay day computed in Leave is the exact same number Payroll reads — there is no second calculation that could silently drift out of sync.

3. Module 1 Recap — Attendance (Existing, In Production)

This section is intentionally brief — Module 1 is already built and running. It is included so the Leave and Payroll sections that follow can reference exactly what already exists rather than assuming it.

3.1 Organization & access model

- **Multi-tenant** — every company is a tenant; every table is tenant-scoped.
- **Roles:** `super_admin` (platform owner) → `tenant_admin` (company owner) → `manager / HR / GM` → `employee`.
- **Privilege system** — beyond role, individual users can be granted specific permissions (e.g. `employee.create`, `attendance.approve`) checked via a single `hasPrivilege()` function. Leave and Payroll extend this exact mechanism with new privilege strings — no new access-control system.

3.2 Attendance verification

- **GPS geofencing** — check-in validated against the tenant's registered office location and radius.
- **WiFi/IP verification** — optional check against the office's public IP.
- **Face verification** — a separate Python microservice (InsightFace/ArcFace) performs liveness detection and identity matching; the Node app only ever handles the resulting score and embedding, never the model itself.
- **QR attendance** — a rotating, tenant-scoped QR code as an alternative check-in path.
- **Work From Home** — an additive attendance mode with its own eligibility policy (allowed roles, weekdays, monthly cap) and registered home-location geofencing.

3.3 Supporting features already in place

- Break sessions with a daily time budget and geofence-aware exit/entry tracking.
- Attendance corrections — an approval-gated workflow, never a silent edit to a log.
- Tenant-configurable holidays and weekend days, used throughout attendance-percentage and (from Module 2 onward) leave-accrual calculations.
- A hash-chained audit ledger for attendance actions — each entry references the previous entry's hash, making tampering detectable. The Financial Ledger in Module 3 is the same idea applied to money.

What Module 1 does not yet have — and what Module 2/3 introduce for the first time: `department/designation/manager` fields on employees, a real (non-mock) policy version history, tenant-level timezone and currency, and bank account details. These gaps are filled in Phase 0, described in Section 8.

4. Module 2 — Leave Management: Features

4.1 Leave Types & Policies

Every tenant defines its own catalog of leave types (Casual, Sick, Earned, Maternity, Comp-Off, etc.), each marked Paid or Unpaid. The rules governing how each type behaves — annual entitlement, carry-forward cap, whether it's encashable, whether unpaid leave beyond balance is allowed — live in a **versioned policy**, not as live-mutable settings. A policy change never rewrites how last year's leave was calculated.

4.2 The Leave Ledger — append-only balance tracking

Leave balance is never a single mutable number. It is the sum of an append-only ledger of credits and debits, exactly the way a bank balance is the sum of transactions, not a field someone edits directly.

Example — one employee's Earned Leave ledger

```
+12.0 Annual Credit (accrual, Jan)
-2.0   Leave Taken           (usage, Mar 14-15)
-3.0   Leave Taken           (usage, Jun 2-4)
+1.0 Manual Adjustment (HR correction, reason logged)
+5.0 Carry Forward (year-end rollover)
-1.0   Expired               (unused balance past cap)
-----
12.0 = CURRENT BALANCE (sum of every row above)
```

4.3 Accrual

- Monthly automated credit, run by the same background scheduler that already powers attendance's daily jobs.
- New joiners are prorated from their join date in their first accrual cycle rather than receiving a full year's credit on day one.

4.4 Leave Requests

- Apply, cancel, and view balance/history — full self-service, no HR ticket needed for the common case.
- **Half-day leave** — morning/afternoon, deducts 0.5 days.
- **Sandwich leave** — an intervening weekend/holiday between two leave dates can optionally count against balance too (tenant policy toggle), reusing the exact working-day-exclusion logic already written for attendance-percentage calculations.
- **Single-level approval** — reuses the existing privilege-gated approval pattern already used for attendance corrections and WFH location changes; no new workflow engine.

4.5 Comp-Off

Comp-off lifecycle

```
Employee works a day flagged Holiday/Weekend in attendance
      |
v
System marks the day ELIGIBLE for a comp-off claim
      |
v
Employee submits a CLAIM (not automatic — avoids false positives
such as a casual login on a day off)
      |
v
```


Manager APPROVES ----> Leave Ledger credited
 |
 v (approval expiry, if unclaimed for N days)
 Credit auto-expires if unused after the policy's configured window

4.6 Restricted / Floating Holidays

A region's government-recognized holidays are rarely all mandatory for every employee — different festivals matter to different communities. This is the standard “restricted holiday” pattern: a pool of optional holiday dates, from which each employee picks only as many as their entitlement allows, and that entitlement is never locked to a specific festival — unused credit can be spent against any other date in the pool.

The pool — extending the existing holidays table

The existing holidays table (currently just `id`, `tenantId`, `date`, `name`) gains four columns:

Column	Purpose
<code>type</code>	'mandatory' (applies to everyone) or 'restricted' (part of the optional pool)
<code>region</code>	State/city; null = tenant-wide. A Tamil Nadu office and a Delhi office don't share the same regional festivals
<code>blockGroup</code>	Groups a multi-day festival together ('diwali-2026') so the UI can show “Diwali (3 days available)” as one labeled group
<code>source</code>	'manual' or 'calendar_sync' — see the sync note below

Entitlement, picking, and the redirect behaviour

Restricted Holiday is simply a new `leave_type` — paid, with an annual entitlement (e.g. 2–3 days). Picking a pool date debits the `leave_ledger` exactly like any other leave usage. Because the entitlement is “N days, spendable against any pool date” rather than “N Diwali days,” the redirect/banking behaviour falls out for free: an employee who skips Diwali entirely and spends the same credit extending Christmas has done nothing special — the pool model already permits it, with no separate swap transaction required.

- Picks are **auto-confirmed**, not routed through manager approval — the tenant already sanctioned the whole pool in advance, so approval adds friction with no real gatekeeping value here.
- Unused credit expires if not spent within a configured window (e.g. 6 months) — the same expiry mechanic already built for comp-off, reused rather than reinvented.
- GET `/api/leave/holiday-pool?month=X` joins pool dates (filtered by the employee's region) against their remaining balance, answering “how many can I take this month” directly.

Keeping the calendar current

“Updated from the calendar” needs an honest, specific answer rather than an assumed live integration: v1 is a yearly CSV a tenant admin uploads per region (`source: 'manual'`), with a public holiday API as a later, pluggable `source: 'calendar_sync'` option once one is actually selected and verified for the regions that matter. Region tagging works today with a plain `region` text field on both `holidays` and `users` — it does not require the full `Branch/Organization-Structure` entity that remains deferred elsewhere in this document.

4.7 Leave Encashment

For leave types marked encashable, unused balance can be converted to money. This is not a standalone feature — it is one additional input to Module 3's payroll-run generation, computed from the same leave ledger and written straight into the Financial Ledger (Section 6) as an earning.

4.8 Loss of Pay (LOP) — the join to Payroll

A single function, `getLopDaysForPeriod(employee, month, year)`, combines two sources: leave requests that exceeded available balance (when the tenant's policy allows unpaid leave), and unexcused absences already visible in attendance logs with no covering leave request or correction. This function is **the** literal integration point between Attendance, Leave, and Payroll — built once in Module 2, consumed by Module 3.

5. Module 2 — Leave Management: Workflows

5.1 Employee workflow

Applying for leave

1. Employee opens Leave tab -> sees current balance per leave type
2. Selects dates, leave type, half-day/full-day, attaches a reason (medical certificate required automatically if Sick Leave > 3 consecutive days)
3. System checks available balance in real time
 - Sufficient balance -> request submitted as PENDING
 - Insufficient, policy allows -> excess days flagged for LOP, still submitted
 - Insufficient, policy blocks -> rejected immediately with a clear reason
4. Employee can CANCEL a pending or future-dated approved request at any time -> cancellation writes a compensating credit, nothing is deleted

5.2 Approver workflow

Reviewing a request

1. Manager/HR (whoever holds 'leave.approve') sees pending requests for their team
 2. Reviews dates against team calendar, attendance history, balance
 3. APPROVE -> Leave Ledger debited, employee notified by email + in-app
- REJECT -> reason recorded, employee notified, balance untouched

5.3 Automated / system workflow

Job	Cadence	What it does
Leave accrual	Monthly	Credits each active employee per their policy's entitlement
Comp-off expiry	Daily	Expires unused comp-off credits past the policy window
Carry-forward / reset	Yearly (year-end)	Rolls unused balance forward up to the cap, or resets/flags for encashment

5.4 The professional workflow — how a real HR team actually runs this

This is the part that matters most for feeling like real company software rather than a form-submission demo. A functioning leave system isn't just "apply / approve" — it has a rhythm across the employee lifecycle and the calendar year.

New joiner

Onboarding

HR creates the employee record -> assigns a leave policy
| (department / role determines which policy applies automatically)
v
First accrual cycle is PRORATED from the join date, not a full year's credit
|
v
Employee sees an accurate starting balance from day one -- no manual HR spreadsheet entry

The ongoing monthly rhythm

A typical month

Throughout the month - employees apply, managers approve, balance ledger updates live

Month-end - HR reviews any unusual patterns (e.g. repeated single-day Sick Leave right after paydays -- a known abuse pattern in real payroll departments)

Month-end - LOP days for the period are now fixed inputs, ready for Payroll to consume (see Section 7.1 for the combined monthly close)

Employee exit

Resignation / termination

Final working day recorded

|

v

Leave Ledger balance frozen -> encashable portion computed per policy

|

v

Encashment amount flows into Final Settlement in Payroll (Section 7.4)

Year-end

- Unused balance either carries forward (up to the policy cap) or is auto-encashed, per each leave type's configured rule.
- A fresh policy version can be published for the new year (e.g. revised entitlement) without touching any historical record — old requests remain tied to the policy version that was active when they were approved.

6. Module 3 — Payroll: Features

6.1 Salary Structures & Components

Each employee's compensation is a **versioned** salary structure (never overwritten — a revision creates a new version, effective from that date), broken into normalized components rather than one JSON blob. This is deliberate: Finance frequently needs to ask “show everyone receiving Fuel Allowance,” which is a plain filtered query against a component table and an awkward one against a JSON array.

6.2 Payroll Component Catalog

Every allowance, deduction, and employer cost — Basic, HRA, PF, Fuel, Insurance, Training budget, and any custom ones a tenant adds — is defined once in a shared catalog, not hardcoded. Each component carries:

- **Category** — Earning / Deduction / Employer Cost / Reimbursement
- **Component Group** — Salary / Allowance / Benefit / Tax / Contribution / Reimbursement / Loan / Bonus (how Finance likes to group a P&L)
- **Formula type** — Flat amount, Percentage of Basic, or Percentage of Gross (deliberately not a full formula language — bounded and predictable)
- **Visibility flags** — Visible on Payslip / Visible to Employee / Included in CTC / Taxable. A laptop-depreciation employer cost, for example, belongs in CTC reporting but must never appear on a payslip — these flags are how that distinction is enforced.

6.3 Statutory Compliance (India)

PF, ESI, Professional Tax, and TDS rates live in a **versioned policy**, exactly like leave policies — a rate change takes effect from a date, and never silently recalculates a prior period. The Loss-of-Pay proration method (actual days in the month vs. a fixed 30-day basis) is an explicit per-tenant setting, since the two methods produce different deductions in a 28-day February versus a 31-day month — a common source of quiet payroll bugs when left implicit.

6.4 The Payroll Run — a controlled processing unit

Payroll is never “generate and done.” It is a stateful record with its own lifecycle, so every run can answer who processed it, when, and whether it can still be changed.

Payroll Run status

DRAFT -> APPROVED -> LOCKED -> POSTED -> ARCHIVED

DRAFT numbers computed, nothing final, freely re-generatable
 APPROVED reviewed and signed off, ledger rows move draft -> approved
 LOCKED no further automatic recompute, even if attendance is corrected later
 POSTED final -- ledger rows count in every dashboard, payslips generated, emailed
 ARCHIVED terminal state, retained for compliance

6.5 The Financial Ledger — single source of truth

Every component of every payroll run — Basic, HRA, PF (employee and employer separately), LOP, a reimbursement, a loan EMI, an encashment payout — becomes one row. Nothing is computed live inside a dashboard; dashboards only aggregate what is already here.

Column	Purpose
componentCode / componentGroup	What this row is — Basic, PF, Fuel — and which broad group it rolls up into

Column	Purpose
category	Earning / Deduction / Employer Cost / Reimbursement
amount, currency	Currency copied from the tenant at write-time, so it never silently changes retroactively
status	draft / approved / posted / reversed — real accounting never deletes, it reverses
metadata (JSON)	The calculation trace — formula, inputs, output — so any number is explainable
payrollRunId, costCenterId	Which run produced this row, and which project/cost center it belongs to
department / designation / role snapshot	Captured at write-time, so a department cost report from last year stays correct even after someone transfers teams
prevHash, hash	Hash-chained, the same tamper-evidence pattern as the existing attendance audit ledger

6.6 Loans & Salary Advances

One table, two use cases: a salary advance is simply a loan with a one-month tenure. `loanType` distinguishes Salary Advance / Personal / Emergency / Education / Relocation. Once approved, the payroll run automatically deducts the EMI each month and decrements the outstanding balance until it reaches zero.

6.7 Reimbursements

Employees submit a claim with a receipt (category, amount, description, upload) against the same component catalog used everywhere else. States: Draft → Submitted → Approved / Rejected → Included in Payroll → Paid, plus Cancelled. Once approved, the next payroll run picks it up automatically and writes the ledger entry — no separate manual payment step.

6.8 Cost Centers

An employee can be allocated across multiple cost centers (e.g. 60% Project A, 40% Project B), versioned with effective dates since allocations change over time. When active, the payroll run splits each component's ledger write proportionally, so “how much did Project X cost us” is answerable directly from the ledger.

6.9 Bank Accounts & Payslips

- Bank details live in their own versioned table (account number, IFSC, holder name), never as a live-mutable field on the employee record — a payroll run snapshots which account was used, so historical payslips remain accurate even after an employee changes banks.
- Payslips store the full breakdown (gross, total employee deductions, total employer contributions, net pay, total employer cost / CTC) as real columns, generated once per employee per posted run, emailed automatically.

6.10 Payroll Validation Report

Before any run can be posted, a validation pass runs automatically and produces a structured report — not a silent pass/fail.

Example validation report

```
198 Employees in this run
196 Valid
2 Missing bank details <- BLOCKS posting for these employees
1 Negative net salary <- BLOCKS posting for this run
```

```
4 Pending attendance corrections (warning, does not block)
3 Pending leave requests overlapping this period (warning)
1 Salary structure missing <- BLOCKS posting for this employee
```

6.11 Payroll Simulation

Because the payroll engine separates *computing* the numbers from *persisting* them, HR can simulate a policy change — “what if PF goes from 12% to 13%?” — and see the impact (total cost before/after, difference, number of employees affected) with zero data written. Same computation function, run twice, diffed.

6.12 Multi-Approval, Accounting/Bank Export, Budgets

- **Multi-approval** — a configurable count of distinct approvals required before a run can lock, without hardcoding specific role names into the schema.
- **Accounting export** — posted ledger entries, grouped by component group, mapped to a tenant's own chart of accounts, exported as a debit/credit journal. Built pluggable (one format now, e.g. generic CSV; additional formats such as Tally added only on real customer demand).
- **Bank export** — a NEFT-style bulk-upload file generated from posted payslips + bank account snapshots. Same pluggable-format principle.
- **Payroll budgets** — a per-department, per-period budget figure compared against actual posted spend, powering a budget-vs-actual dashboard.

7. Module 3 — Payroll: Workflows

This section is the heart of “how a company professional workflow would be” — the monthly payroll close is a real, well-established business process at any company, and the system is designed to mirror it exactly rather than reduce it to a single “Generate Payroll” button.

7.1 The monthly payroll close — a full-month timeline

This is how an HR/Finance team actually runs a month, from the first working day through bank transfer and accounting close. Dates below are illustrative for a typical Indian monthly-payroll company; the actual cutoffs are tenant-configurable.

When	What happens	Who
Day 1 - 25	Employees clock in/out; leave is applied and approved; attendance corrections are requested and approved as needed	All employees, Managers
~Day 25	Attendance correction window closes for the period. Late corrections roll into next month's retroactive adjustment, not this month's run	HR
~Day 26	Payroll run created in DRAFT. System pulls: current salary structures, active statutory policy, LOP days (Section 4.7), approved reimbursements, active loan EMIs, any encashment due this cycle	HR / Payroll admin
Day 26 - 27	Validation Report reviewed (Section 6.10). Any hard-blocking issues (missing bank details, negative pay, missing salary structure) resolved before proceeding	HR
Day 27 - 28	Numbers reviewed via drill-down — total cost, department cost, individual anomalies (a duplicate component, an unusually large jump vs. last month) checked against the ledger's calculation metadata	HR, Finance
Day 28	Run APPROVED. Required sign-off count satisfied. Ledger rows move draft -> approved	Whoever holds payroll.approve
Day 29	Run LOCKED. No further automatic recompute is possible from this point — any further attendance correction becomes a retroactive adjustment landing in *next* month's run, never edits this one	HR
Day 30 / 1st	Run POSTED. Ledger rows become final and count in every dashboard. Payslips generated and emailed. Bank export file generated and uploaded to the bank portal for NEFT transfer	Finance
1st - 3rd	Accounting export generated for the finance team's books; PF/ESI/PT/TDS statutory filings prepared from the same posted ledger	Finance

7.2 Employee-facing workflow

- View current CTC breakdown (gross, deductions, employer cost) at any time — not just on payday.
- Download payslips and view payment history once posted.
- Submit reimbursement claims with receipts; track claim status through to payment.
- Request a salary advance or loan; track EMI deductions against outstanding balance.
- See exactly why a number is what it is — clicking a payslip line shows the same calculation trace HR sees (Section 6.5's ledger metadata), just scoped to that employee.

7.3 HR / Finance workflow

This is the Section 7.1 timeline from the operator's seat — the actions available at each stage of a run's lifecycle:

Actions available per Payroll Run status

DRAFT -> re-generate (if data changed), review drill-down, run validation
 APPROVED -> lock, or revert to draft if an issue is found before locking
 LOCKED -> post, or (rare) an authorized unlock with reason logged to the audit trail
 POSTED -> generate exports (bank file, accounting journal), view-only from here

7.4 New joiner → Payroll

Bringing a new employee into payroll

HR creates the employee record, assigns department/designation/manager
 |
 v
 HR creates the employee's first Salary Structure (effective from join date)
 |
 v
 Employee optionally submits bank account details (self-service, same optional-until-completed pattern as KYC)
 |
 v
 First payroll run after joining automatically prorates pay for a partial month

7.5 Employee exit → Final Settlement

Resignation / termination

Last working day confirmed
 |
 v
 System computes final settlement:
 + Pending salary up to last working day (prorated)
 + Leave encashment (unused balance x per-day rate, per policy)
 - Outstanding loan balance (recovered in full, or per exit policy)
 + Any pending approved reimbursements
 - Notice-period shortfall, if applicable
 |
 v
 Final settlement run generated -> same validation, approval, posting pipeline as a normal run, just flagged as a one-time final settlement for that employee

7.6 Year-end

- Leave carry-forward/expiry runs (Section 5.3) feed encashment amounts into the appropriate payroll run.
- Updated statutory rates for the new financial year are published as a new policy version, effective from a specific date — every payslip generated afterward automatically uses the new rate; every payslip already posted stays exactly as it was.
- Annual cost reports and department-level drill-downs (Section 6.5's Financial Ledger, aggregated over a full year) become available once a full year of posted ledger data exists.

7.7 Payroll Correction / Retro-Pay

What actually happens when a **posted** payroll run turns out to have an error — answered directly rather than left implicit in the run-lifecycle description:

An error is found in a run that's already posted

Error discovered (wrong attendance correction applied late, a missed reimbursement,

```
a salary structure that should have been effective a month earlier)
|
v
The POSTED run is never edited -- its ledger rows stay exactly as they were,
status stays 'posted' (Section 6.5's reverse-not-delete principle)
|
v
A retroactive_adjustment ledger entry is created instead:
  - references the ORIGINAL payrollRunId in its metadata
  - amount is the DELTA only (what should have been paid, minus what was),
not a restatement of the full figure
  - status: 'draft', same approval gate as any other ledger write
|
v
Approved through the SAME payroll.approve privilege as a normal run --
no separate 'correction approver' role invented
|
v
The delta lands in the NEXT payroll run's net pay, clearly labelled as an adjustment
on that payslip, with a reference back to which month it corrects
```

This is not a new mechanism — it's the retroactive-adjustment behaviour already specified for attendance-correction-after-lock (Section 6.4) generalized to any post-posting error, using the same ledger, the same approval privilege, and the same audit trail.

8. Cross-Module Integration — The Foundation Layer

Every feature above depends on a small set of shared building blocks, added once and reused by both new modules rather than duplicated. This is what makes the platform coherent instead of three modules that happen to share a login.

8.1 Tenant-level timezone & currency

Two new fields on the existing tenant record. Since each tenant is a distinct company, different clients can genuinely be in different countries — this is not a foreign-exchange conversion engine, just what a tenant's numbers are denominated and displayed in, and which clock its day/week/month boundaries are computed against. This also fixes a latent bug: date-boundary logic (which weekday is “today,” month-boundary attendance calculations) previously used the server's local clock rather than the tenant's.

8.2 Real policy versioning

A single, reusable mechanism: every policy (leave rules, statutory rates) is a dated, immutable version, not a live-mutable setting. This did not exist anywhere in the platform before — it is introduced once, generically enough that attendance policies could adopt the same mechanism later if desired, without needing a second, differently-built version system.

8.3 Role & privilege extension

No new access-control mechanism — new privilege strings slot into the exact `hasPrivilege()` system already gating attendance actions:

New privilege	Grants
leave.approve	Approve/reject leave requests and comp-off claims
leave.manage	Configure leave types and publish leave policy versions
payroll.run	Generate, review, and approve payroll runs
payroll.view	View payroll reports and dashboards without run authority
reimbursement.approve	Approve/reject reimbursement claims
loan.approve	Approve loan and salary advance requests

8.4 Scheduler — organized by cadence

Cadence	Jobs
Daily	Comp-off expiry check, end-of-day attendance auto-checkout (existing)
Monthly	Leave accrual, payroll run auto-draft reminder
Yearly	Leave carry-forward / reset, statutory-rate policy renewal prompt

8.5 The two ledgers, side by side

	Leave Ledger	Financial Ledger
Tracks	Days of leave balance	Money — earnings, deductions, employer costs
Written by	Accrual cron, leave approval, comp-off, encashment trigger	Payroll run generation, reimbursements, loans

	Leave Ledger	Financial Ledger
Never does	Store a mutable "balance" field	Delete a row — always reverses instead
Read by	Leave balance screens, LOP calculation	Every payroll dashboard, cost report, drill-down

Both ledgers exist for the same reason the existing attendance audit ledger exists: a number is only trustworthy if you can show exactly how it was produced. Future modules (assets, training budgets, recruitment costs) are expected to write into the same Financial Ledger rather than invent a fourth ledger of their own.

9. Notification Engine — Systematic, Every Time

Today, notifications are an afterthought: the `notifications` table exists but is written to from a single place in the entire backend, there is no in-app notification UI anywhere in the frontend, and email is sent through twelve separate hand-written functions in `mail.ts` — one per event type. Every new event has meant writing a new function from scratch. That does not scale to two more modules' worth of events, and it means notification coverage depends on a developer remembering to add it per route rather than the system guaranteeing it.

The fix is the same discipline applied everywhere else in this document: one shared mechanism, reused by every module, instead of a bespoke implementation per feature.

9.1 One dispatch function, both channels, always together

The systematic pattern – replaces 12 bespoke email functions

```
notify({
  tenantId,
  recipients: [userIds] OR a privilege string (e.g. 'leave.approve')
    -- resolved via the EXISTING getUsersWithPrivilege() pattern,
  no new recipient-resolution logic
  type: 'leave.approved' -- one short, namespaced event key
  data: { employeeName, dates, ... } -- fills a template for this type
  link: '/leave/requests/42' -- where the notification redirects to
})
```

- |
- v
1. Writes ONE row per recipient to ``notifications`` (in-app)
 2. Renders the subject/body for ``type`` from a shared template registry and sends it through the EXISTING generic `sendEmail()` -- not a new bespoke function per event
 3. Both channels fire from the same call -- never a case where the in-app entry exists but the email didn't, or vice versa

A new Leave or Payroll event becomes one new entry in a template registry (subject + body for that type), not a new exported function. This is what “systematic” means concretely: coverage is a data-table lookup, not something a developer has to remember to wire up correctly on every new route.

9.2 Schema — extending, not replacing, the existing table

Column	Status	Purpose
id, userId, title, message, isRead, createdAt	Existing	Unchanged
tenantId	New	Currently missing — needed for tenant-scoped queries and cleanup
type	New	The namespaced event key (e.g. <code>leave.approved</code> , <code>payroll.posted</code>) driving the template and the icon shown in-app
link	New	The redirect path — what makes the notification center clickable rather than read-only
emailSent	New	Whether the email channel was also dispatched for this row, for auditability

9.3 Every event, by recipient — the complete coverage matrix

This is the systematic part made concrete: every state-changing action across Attendance, Leave, and Payroll that should notify someone, listed once, so coverage can be checked against this table rather than trusted to memory.

Recipient	Notified when...	Redirects to
Employee	Leave request approved / rejected	That leave request's detail view
Employee	Comp-off credited, or expiring soon	Leave balance screen
Employee	Payslip posted and ready	That payslip
Employee	Salary revision becomes effective	Compensation summary
Employee	Loan / advance approved or rejected; each EMI deducted	Loan detail / EMI schedule
Employee	Reimbursement claim approved, rejected, or paid	That claim
Employee	Attendance correction approved / rejected (existing)	That correction
Employee	Bank details missing, payroll cutoff approaching	Bank details form
Manager	A team member has a pending leave request or comp-off claim	Pending approvals queue
Manager	A team member's attendance correction needs review (existing)	Pending corrections queue
HR	An employee is missing a salary structure before a payroll cutoff	That employee's compensation tab
HR	Comp-off credits are expiring unclaimed, tenant-wide	Comp-off report
HR	A payroll run's Validation Report has blocking errors	That run's validation report
HR / Finance	A payroll run is ready for review	That run's drill-down view
Finance	A payroll run is approved and awaiting lock/post	That run
Finance	Bank export or accounting export file is ready	The export download
Finance	A department's spend is approaching or exceeding its budget	Budget dashboard for that department
Tenant Admin	A payroll run has posted — summary of total cost	Payroll run summary

9.4 In-App Notification Center

A bell icon added to the existing dashboard chrome (`PortalShell.tsx` / `BottomNav.tsx`), showing an unread-count badge sourced from `GET /api/notifications/unread-count` — polled the same way the existing QR check-in display already re-fetches on an interval, not a new real-time infrastructure the app doesn't otherwise have.

Clicking a notification — the redirect behaviour

Bell icon (badge: 3)

| click

v

Dropdown/panel: most recent notifications, unread visually distinct
'Leave request from Priya - Pending your approval' 2m ago
'Payslip for April is ready' 1h ago
'Payroll Validation: 2 employees missing bank details' 3h ago
| click one entry

v

1. Marks that notification read (POST /api/notifications/:id/read)
2. Navigates (React Router) directly to its `link`
-- e.g. the payslip notification opens that exact payslip,
not a generic 'Payroll' landing page

- GET /api/notifications — paginated, most recent first
- GET /api/notifications/unread-count — powers the badge
- POST /api/notifications/:id/read and /read-all
- All four reuse the existing authenticate middleware — no new auth mechanism.

9.5 Migration path for the twelve existing email functions

The existing attendance-related functions in `mail.ts` (`sendWfhDecisionEmail`, `sendAttendanceCorrectionEmail`, and the rest) keep working unchanged — nothing breaks on day one. Every **new** Leave/Payroll event is built through `notify()` from the start. The existing functions are folded into the same template registry opportunistically, whenever that specific email is next touched for another reason — not as a risky big-bang rewrite of working code.

10. Feature Entitlement Cascade

This is the mechanism that governs whether any capability described elsewhere in this document — a business module (Restaurant, Prime Cost), a Leave/Payroll feature, or an administrative action (hiring, granting privileges) — is even visible to a given user. Nothing in this platform should be hardcoded to a role string. Every capability, including basic administrative ones, is a toggleable entitlement, delegated downward through a hierarchy where no level can grant more than it itself holds.

10.1 The hierarchy and the one rule that secures it

Super Admin -> Tenant Admin -> GM -> Manager -> Employee

Super Admin

defines the platform's full feature catalog, enables/disables features globally

|

v

Tenant Admin

receives an approved subset at onboarding, delegates any subset of it to GMs

|

v

GM -> Manager -> Employee

same rule repeats at every level, no special-casing per role

THE INVARIANT:

No user can grant a feature or action they do not currently possess.

That single rule secures the entire delegation chain, at every level, forever.

Note what this hierarchy is *not*: it is not an organizational reporting chain. “GM creates Manager” is an administrative action gated by an employee.create entitlement — it does not imply the Manager reports to that GM for HR purposes. Who approves whose leave and who can see whose attendance is a separate concern (managerId/departmentId, deferred org-structure work) and is deliberately not conflated with this delegation mechanism.

10.2 One catalog — modules and administrative actions, identical treatment

There is no separate “admin permissions” system running alongside a “feature flags” system. Everything is a row in the same catalog:

type	Example code	Meaning
module	attendance, leave, payroll, restaurant, prime_cost	A whole capability area is available
action	employee.create, privilege.assign, role.edit, salary.run, shift.assign	A specific administrative or operational action is permitted

The type column exists purely so the toggle-picker UI can group “Modules” separately from “Administrative Actions” — it changes nothing about how entitlement is checked or delegated. Hiring an employee, assigning a privilege, and turning on the Restaurant module are, to this system, the exact same kind of thing.

10.3 Schema

Table	Purpose
features	The platform catalog: code, name, type (module/action), category, isGloballyEnabled
tenant_features	tenantId, featureId, approvedBy, enabledAt, disabledAt — which catalog features a tenant is approved for, with full enable/disable history (same pattern as user_features below)

Table	Purpose
user_features	userId, featureId, grantedBy, grantedAt, revokedAt — who has what, and the full audit trail (never delete a row; revoke by timestamp, insert fresh on re-grant)
feature_dependencies	featureId, dependsOnFeatureId — a bounded dependency graph, not a rules engine

tenant_features is not a single mutable boolean — every enable/disable is a new row (or a stamped disabledAt on the current one, with a fresh row on re-enable), so a question like “why wasn’t payroll running in February” has a direct, queryable answer: the Restaurant module (which Payroll depended on) was disabled Feb 10 and re-enabled Mar 5. This is the same discipline as everything else in this document — never overwrite state that has historical value.

user_features is deliberately normalized, not a JSON array on the user record — easier revocation, easier “who has X” queries, and the grantedBy/grantedAt/revokedAt columns are the audit trail on their own, no separate log table required. The effective feature list for a session is computed once at login and cached (JWT or server-side session) so the hot runtime check stays O(1) regardless of how normalized the underlying storage is.

10.4 Onboarding — deny-by-default, not opt-out

Tenant onboarding flow

```
Tenant Admin signs up
    |
v
Selects the features they want from the platform's full catalog
    |
v
Super Admin reviews the request -> approves fully, or prunes specific items
    |
v
Tenant receives exactly the approved set -- nothing is on by default
```

Deny-by-default rather than everything-on-by-default is a deliberate choice: this system gates capabilities like `privilege.assign` and `salary.run`, and relying on Super Admin to notice and disable an inappropriate default is a weaker security posture than an explicit, reviewed allowlist. This manual review flow is also the direct precursor to automatic plan-based provisioning once pricing tiers exist (Basic -> 25 features, Professional -> 60, Enterprise -> all) — same mechanism, the approval step just becomes automatic.

This request-and-approve flow is not onboarding-only. The identical mechanism handles a tenant asking for an additional feature six months in — `POST /api/tenant/feature-requests` at any time, reviewed the same way. There is exactly one request/approve code path, used at signup and reused for every request after it, not a special first-time-only step.

10.5 Feature dependencies — confirm and cascade, both directions

Some features are meaningless without another — Prime Cost Dashboard depends on Payroll data; Restaurant Analytics depends on the Restaurant module being on. `feature_dependencies` makes this explicit and self-describing, rather than a fact every future engineer has to independently rediscover.

Enabling and disabling with dependencies

```
ENABLE Prime Cost Dashboard
  -> depends on: Payroll, Restaurant
  -> UI shows: "Enabling Prime Cost will also enable Payroll and Restaurant" -> confirm

DISABLE Payroll
  -> Prime Cost Dashboard and Salary Run both depend on it
  -> UI shows: "This will also disable Prime Cost Dashboard and Salary Run" -> confirm
```

Both directions are confirm-and-cascade — never a silent auto-enable, and never a hard block with no path forward. As the catalog grows from the current handful of features toward hundreds, this table is what keeps dependency logic in data instead of scattered conditionals across the codebase. One integrity rule the write path enforces: inserting a `feature_dependencies` row that would create a cycle (A depends on B, B depends on A, directly or transitively) is rejected outright — a standard topological-sort check on write, not a new subsystem. Dependencies stay hard-only (“requires”, not “works better with”) until a real feature actually needs the softer distinction — same bounded-scope discipline as the formula types in Section 6.1.

10.6 Migrating off the existing privileges array

The existing `users.privileges` JSON array (Module 1, live in production today) is not exempt from this system — leaving it as a permanent parallel mechanism would break the “one catalog, one rule” principle this whole design exists to establish. The migration is staged, not a rip-and-replace:

- Staged migration, verified the same way as every other production change
1. Build `features` / `user_features` / `feature_dependencies` fresh -- zero risk, nothing depends on them yet

2. One-time data migration: seed the catalog with every privilege string already in use; seed `user_features` from every existing user's current privileges + role defaults

3. Rewrite `hasPrivilege()` / `getEffectivePrivileges()` INTERNALS only -- signatures stay identical, so none of the ~74 existing route handlers that call them need to change

4. Verify: typecheck, tests, a smoke script against real Postgres confirming login and privilege-check behaviour is byte-identical before and after

5. Only then retire the `privileges` column

10.7 The runtime check — one function, everywhere

- `hasFeature()` -- the only check any route or UI element makes
- `hasFeature(user, featureCode):`

1. Is the feature globally enabled? -- Super Admin's kill switch

2. Has the tenant been approved for it? -- Tenant Admin's cascade

3. Does the user hold it (own grant or role default)? -- the existing, now-normalized privilege mechanism
- Every screen, every API route, checks only `hasFeature("salary.run")`.
No role checks. No tenant checks. No hardcoding, anywhere.

On the frontend specifically, this replaces the hardcoded role-string checks that exist today (e.g. `canClockIn(role)`, `canSeeDashboard(role)` in `AdminApp.tsx`) with one dynamic list, fetched once at login (`GET /api/me/features`) and checked by membership everywhere nav items, dashboard cards, and routes decide what to render.

10.8 Every dynamic surface, not just navigation

“Nothing hardcoded” is stated as a principle in 10.7 — this is what it means concretely, surface by surface. Each one is a render loop over the same `GET /api/me/features` list, not a separate mechanism per surface:

Surface	Driven by feature membership
Sidebar / navigation	A nav item exists only if its feature code is in the user's list — no route is ever reachable through the UI without also passing <code>hasFeature()</code> on the API side
Dashboard cards / widgets	A card renders only if its underlying feature (and, per Section 10.5, that feature's dependencies) is present

Surface	Driven by feature membership
Reports	A report is itself a feature code — “Payroll Cost Report” doesn't render for a user without payroll.view, same as any module
API surface	Every route handler's first check is hasFeature() — the API is the enforcement point; the UI hiding an element is convenience, not the security boundary

The API check is what actually matters — a hidden sidebar item is a UX nicety, not protection. If `hasFeature()` isn't checked server-side on the route itself, hiding it client-side is cosmetic only.

10.9 The full lifecycle, end to end

From platform definition to a running check, in one picture

Super Admin defines a feature in the catalog, sets `isGloballyEnabled`

|

v

Tenant Admin requests it (onboarding, or any time after -- Section 10.4)

|

v

Super Admin approves (or prunes) the request

|

v

Tenant Admin delegates a subset to a GM

|

v

GM delegates a subset of THEIR set to a Manager (same rule, every level)

|

v

Manager delegates a subset to an Employee

|

v

Every grant/revoke at every step writes a `user_features` / `tenant_features` row

|

v

`hasFeature()` reads the cached, effective result -- sidebar, dashboard, API, all of it

10.10 Deliberately deferred — SaaS commercial features, not EMS features

A distinct category worth naming explicitly, so it isn't mistaken for a gap: feature **versioning** (running Prime Cost v1 and v2 side by side for different tenants), feature **usage analytics** (adoption dashboards for the platform owner), **vertical packs/templates** (bundling a preset like “Restaurant Starter” for faster onboarding), feature **expiry**, and **trial periods**. These are real, standard capabilities of a mature multi-tenant SaaS product — but they are product/commercial concerns (billing, packaging, growth) layered on top of this entitlement system, not requirements of the EMS itself. None of them are needed for Phase 0–5 to work correctly, and building them now would mean designing for a subscription/billing model that doesn't exist yet. They plug into the schema here cleanly whenever that becomes a real, separate requirement — features already has room for a `planRequirement` field, and vertical packs are just a named, pre-selected list of feature codes on top of the existing catalog, not a new mechanism.

11. Organization Setup & Bulk Operations

Three workflows that don't belong specifically to Leave or Payroll, but that every tenant needs early and at scale: standing up a brand-new company on the platform, importing a workforce all at once rather than one employee at a time, and offboarding at the same scale. Each reuses infrastructure already specified elsewhere in this document.

11.1 “Day 0” — new tenant setup

From signup to a working team

```
Tenant Admin signs up -> requests features (Section 10.4)
    |
v
Super Admin approves -> tenant has its entitled feature set
    |
v
Tenant Admin configures:
  - Departments / designations (flat fields on users -- Section 10.1's deferred
org-structure note applies; Branches specifically remain out of scope until
that entity is actually built)
  - Shift / attendance policy (existing Module 1 config)
  - Leave policy -- first policy_version, domain='leave' (Section 4.1)
  - Payroll policy -- statutory rates, domain='payroll' (Section 6.3)
  - Salary structure templates (Section 6.1)
    |
v
Tenant Admin creates GMs/Managers, delegating a subset of the tenant's features
    |
v
First employees invited -- individually, or via Bulk Onboarding (11.2) for scale
```

Nothing here is a new mechanism — this is a guided sequencing of steps that already exist individually elsewhere in this document, presented as one coherent “getting started” path rather than left for a new tenant admin to discover on their own.

11.2 Bulk Onboarding

Importing a workforce, not one employee at a time

```
HR uploads a CSV/Excel file (name, email, role, department, designation, ...)
    |
v
Row-level validation BEFORE any writes:
  - required fields present, email format valid, role is a real role,
no duplicate emails within the file or against existing employees
    |
v
Error report shown -- exactly which rows failed and why, before committing anything
    |
v
HR chooses: fix and re-upload, or proceed with valid rows only
    |
v
Valid rows are created; invited employees get the standard activation email
    |
v
Import summary: N created, M skipped, full list of skipped rows with reasons
```

Partial success by default, not all-or-nothing — a single bad row (a typo'd email, a duplicate) shouldn't block 499 valid ones. Each created employee gets whatever feature defaults the importing user is entitled to delegate (Section 10), same as one-at-a-time creation — bulk import is a faster path through the existing creation flow, not a separate, less-audited one.

11.3 Bulk Offboarding & the Exit Checklist

Extends the individual Final Settlement workflow (Section 7.5) with the surrounding administrative steps, both for one departing employee and for a batch (e.g. a seasonal team's contract ending together):

- **Disable login** — immediate, via the existing session-revocation mechanism (JWT session invalidation already used for password resets).
- **Final Settlement** — runs exactly as specified in Section 7.5.
- **Exit checklist** — a configurable list of steps a manager/HR ticks off (access cards returned, equipment returned, exit interview completed). Kept deliberately lightweight: a checklist of boolean items logged to the audit trail, not a dependency on an Assets Management module that doesn't exist yet — “equipment returned” is a checkbox HR confirms, not a system that tracks serialized asset inventory.
- **Archive, not delete** — the employee record is marked inactive, never removed; attendance/leave/payroll history remains queryable, consistent with the reverse-not-delete principle applied everywhere else in this document.

12. Roles & Responsibilities Matrix

Who can do what across Leave and Payroll, mapped onto role. Read this as the **typical default bundle** a role receives, not a hardcoded gate — per Section 10, the actual mechanism is the feature entitlement cascade, and any of these can be individually granted or withheld per user regardless of role. A checkmark below means “this is what role defaults to,” not “this is the only way it can be.”

Action	Employee	Manager	HR	Tenant Admin
Apply / cancel own leave	Yes	Yes	Yes	-
Approve leave requests	-	Team only	Yes	Yes
Configure leave policy	-	-	Yes (if granted)	Yes
View own payslip / CTC	Yes	Yes	Yes	-
Submit reimbursement claim	Yes	Yes	Yes	-
Approve reimbursement / loan	-	If granted	Yes	Yes
Generate payroll run	-	-	If granted	Yes
Approve / lock / post payroll run	-	-	If granted	Yes
View department cost dashboards	-	Own team	Yes	Yes
Configure statutory policy rates	-	-	-	Yes
Export bank / accounting files	-	-	If granted	Yes

13. Data Model Summary

Every new table introduced across Modules 2 and 3, for quick reference. All are additive — no existing attendance table is modified in a breaking way; only two nullable columns are added to the existing employee record (department, designation, manager reference).

13.1 Foundation (shared by both modules)

Table	Purpose
financial_ledger	The single source of truth for every earning, deduction, and employer cost
policy_versions	Dated, immutable policy versions — leave rules, statutory rates
payroll_runs	One row per payroll cycle; the controlled processing unit (Section 6.4)
cost_centers	Projects/centers an employee's cost can be allocated against
notifications (extended)	In-app + email dispatch for every event across all three modules (Section 9)
features	Platform-wide feature/action catalog, Super Admin managed (Section 10)
tenant_features	Which catalog features a tenant has been approved for (Section 10)
user_features	Normalized per-user grants with a built-in audit trail (Section 10)
feature_dependencies	Which features require which others (Section 10)

13.2 Leave Management

Table	Purpose
leave_types	Tenant's catalog of leave categories
leave_ledger	Append-only credit/debit history — balance is its sum
leave_requests	Applications for leave, with approval status and computed LOP days
comp_off_claims	Eligibility -> claim -> approval trail for compensatory off
holidays (extended)	Existing table +type/region/blockGroup/source — mandatory vs. restricted-pool holidays (Section 4.6)

13.3 Payroll

Table	Purpose
salary_structures / salary_structure_components	Versioned per-employee compensation, normalized into rows
payroll_components	Catalog of every allowance/deduction/employer cost, with visibility flags
employee_bank_accounts	Versioned bank details, snapshotted per payroll run
payslips	Per-employee, per-run summary — gross, deductions, employer cost, net
loans	Loans and salary advances (one table, distinguished by loanType)
reimbursement_claims	Employee expense claims through to payment

Table	Purpose
employee_cost_center_allocations	Versioned % split of an employee's cost across projects
accounting_gl_mappings	Tenant's chart-of-accounts mapping for journal export
payroll_budgets	Budget figures compared against actual posted spend
payroll_run_approvals	Sign-off records when multi-approval is configured

14. Phased Implementation Roadmap

The build order that gets to a working, demoable system fastest — each phase leaves the application fully functional, nothing is ever half-built in production.

Phase	Delivers
0 — Foundation	Financial Ledger, Policy Versions, tenant timezone/currency, department/designation/manager fields, and the Feature Entitlement Cascade (Section 10) — gating everything built in every phase after this one. No user-facing UI yet.
1 — Leave Management	Leave Ledger, types, requests, accrual, approval, sandwich/half-day leave, Restricted/Floating Holidays, the LOP calculation function
2 — Payroll	Salary structures, statutory calculation, the full run lifecycle, payslips
3 — Dashboards & Analytics	Drill-down cost reporting, plus an entitlement Audit Dashboard (who granted/revoked which feature, when — a UI screen over the Section 10.3 grant history, no new backend) — both mostly free once the ledger and entitlement tables exist, near-zero new schema
4 — Extended Finance Features	Comp-off, encashment, loans/advances, reimbursements, cost-center allocation, bank/accounting export, validation report, simulation, multi-approval, budgets
5 — Vertical Extensions (optional)	Restaurant-specific features (Appendix A) — built only if this deployment is actually a restaurant, gated by the same entitlement cascade as everything else

Feature Entitlement moving into Phase 0 is deliberate, not incidental: gating a capability after it's already built means retrofitting every route and UI element after the fact; gating it from the start means every module built in Phases 1 through 5 is born behind a toggle. Section 10's staged migration off the existing privileges array (10.6) should also happen during Phase 0, while the least code depends on the old shape.

Deliberately out of scope until a real requirement demands them: multi-country taxation, union/collective-bargaining agreements, stock grants/ESOPs, and AI-flavored forecasting or fraud-detection features. All deterministic validation (negative pay, missing bank details, duplicate components) is built as real rule-based checks, not machine learning — the distinction matters, since these are reliable, auditable, testable checks rather than a black box.

15. Design Principles — What Ties This Together

- **One ledger, many views.** Every dashboard is a filtered aggregate query over the Financial Ledger — no bespoke calculation logic duplicated per screen.
- **Reverse, never delete.** Corrections, cancellations, and adjustments write new rows; nothing is silently edited or removed. This is true of attendance corrections today, and of every new leave/payroll record.
- **Snapshot at the moment of truth.** Department, designation, salary, and bank account are captured at the time a transaction happens, so historical reports remain accurate even after an employee transfers teams, gets a raise, or changes banks.
- **Policies are dated, not live settings.** A rate or rule change takes effect from a date and is retrievable exactly as it was for any historical calculation.
- **Reuse the access-control and scheduling systems that already exist.** No second RBAC system, no second background-job runner — new privilege strings and new scheduled jobs slot into what Module 1 already built and proved in production.
- **Bounded, not speculative.** Formula types are Flat / % of Basic / % of Gross, not a general expression language. Export formats are pluggable but only one is built until a real customer needs another. The biggest risk to a system like this is over-engineering for hypothetical requirements instead of the one in front of you.

Appendix A — Restaurant Vertical Extension (Optional)

Everything in this appendix is **optional and gated by the Feature Entitlement Cascade (Section 10)** — built only for deployments that are actually restaurants, invisible to every other tenant. None of it is part of the core Phase 0–4 roadmap; it is Phase 5, pursued only if there is a real restaurant customer to build it for. Every idea below extends infrastructure already specified elsewhere in this document rather than introducing a second architecture.

A.1 Prime Cost / Labor-Cost-to-Sales

The one metric a restaurant owner or GM checks obsessively, and the headline reason this vertical is worth pursuing at all. One small table — `daily_sales` (`tenantId`, `date`, `mealPeriod`, `salesAmount`, `coversCount`) — manager-entered, no POS integration required for v1 — makes labor cost % of sales a live aggregate query over the Financial Ledger that already exists, broken down by meal period, station, or shift exactly like every other ledger-backed dashboard in this document.

A.2 Tip-Out Distribution Chains

Restaurants run codified tip-out chains, not simple pooling — a server pays a fixed % of their tips to the bar, a % to bussers, sometimes a % to kitchen. Modeled as a new `payroll_components` formula type (`tip_out_chain`), computed from each server's shift totals, writing one Financial Ledger row to every recipient — the same component-and-ledger pattern already used for every other payroll component, just modeling a chain instead of a flat split.

A.3 Meal-Period & Covers-Driven Scheduling

Staffing follows brunch/lunch/dinner/late-night demand, using historical covers plus today's reservation count rather than generic occupancy — the restaurant-specific version of demand-aware rostering.

A.4 Emergency Shift-Swap Broadcast

A no-show 30 minutes before dinner service broadcasts an urgent cover request to every off-duty employee certified for that station — first to accept gets it, manager confirms. One new event type on the existing Notification Engine (Section 9), race-to-accept instead of passive read.

A.5 Station Certification-Gated Scheduling

Grill, sauté, pastry, bar, and wine service are internal skill badges, gated the same way as statutory certifications (food handler, alcohol service) — an employee cannot be scheduled into a station they are not marked trained for, and expiring statutory certifications block scheduling entirely, with the existing notification engine firing 30/15/7-day reminders before a lapse.

A.6 Split-Shift Premium Automation

Lunch-then-dinner split shifts routinely trigger split-shift premium pay under local labor law and are commonly miscalculated by hand. Detecting the split from attendance timestamps and applying the correct premium is one more formula in the payroll component system, not new infrastructure.

A.7 Comp / Void / Walkout ↔ Shift Accountability

An unusual spike in comps, voided items, or a walked table links back to the exact server, shift, and table for manager review — the same audit-ledger traceability principle applied everywhere else in this document, aimed at the events that matter on a restaurant floor.

A.8 Staff Meal as a Tracked Benefit

The free “family meal” before/after a shift is a real, usually untracked, employer cost. One more Financial Ledger component (`staff_meal`, employer cost, non-taxable by default) — consistent with “everything that costs the company money is a ledger entry.”

This specification reflects the full feature set and workflow design discussed and refined across the planning process for Smart Teams EMS Modules 2 and 3, integrated with the existing, production Module 1 attendance system.

LEAVE MANAGEMENT + PAYROLL

Features, Workflows & Integration Architecture

A complete specification of Module 2 (Leave) and Module 3 (Payroll) — every feature, every workflow, and exactly how both connect into the existing attendance system, Module 1, as one continuous employee-cost platform.

Prepared for internal engineering & product planning

Covers: Phase 0 (Foundation) through Phase 4 (Extended Finance Features)

Document scope: architecture, database schema, business workflows, role responsibilities

Table of Contents

Table of Contents	2
1. Executive Summary	5
2. System Overview — Three Modules, One Platform	6
2.1 The dependency chain	6
2.2 What makes this different from a typical student/college EMS	6
3. Module 1 Recap — Attendance (Existing, In Production)	7
3.1 Organization & access model	7
3.2 Attendance verification	7
3.3 Supporting features already in place	7
4. Module 2 — Leave Management: Features	8
4.1 Leave Types & Policies	8
4.2 The Leave Ledger — append-only balance tracking	8
4.3 Accrual	8
4.4 Leave Requests	8
4.5 Comp-Off	8
4.6 Restricted / Floating Holidays	9
4.7 Leave Encashment	9
4.8 Loss of Pay (LOP) — the join to Payroll	9
5. Module 2 — Leave Management: Workflows	11
5.1 Employee workflow	11
5.2 Approver workflow	11
5.3 Automated / system workflow	11
5.4 The professional workflow — how a real HR team actually runs this	11
6. Module 3 — Payroll: Features	13
6.1 Salary Structures & Components	13
6.2 Payroll Component Catalog	13
6.3 Statutory Compliance (India)	13
6.4 The Payroll Run — a controlled processing unit	13
6.5 The Financial Ledger — single source of truth	13
6.6 Loans & Salary Advances	14

6.7 Reimbursements	14
6.8 Cost Centers	14
6.9 Bank Accounts & Payslips	14
6.10 Payroll Validation Report	14
6.11 Payroll Simulation	15
6.12 Multi-Approval, Accounting/Bank Export, Budgets	15
7. Module 3 — Payroll: Workflows	16
7.1 The monthly payroll close — a full-month timeline	16
7.2 Employee-facing workflow	16
7.3 HR / Finance workflow	16
7.4 New joiner → Payroll	17
7.5 Employee exit → Final Settlement	17
7.6 Year-end	17
7.7 Payroll Correction / Retro-Pay	17
8. Cross-Module Integration — The Foundation Layer	19
8.1 Tenant-level timezone & currency	19
8.2 Real policy versioning	19
8.3 Role & privilege extension	19
8.4 Scheduler — organized by cadence	19
8.5 The two ledgers, side by side	19
9. Notification Engine — Systematic, Every Time	21
9.1 One dispatch function, both channels, always together	21
9.2 Schema — extending, not replacing, the existing table	21
9.3 Every event, by recipient — the complete coverage matrix	22
9.4 In-App Notification Center	22
9.5 Migration path for the twelve existing email functions	23
10. Feature Entitlement Cascade	24
10.1 The hierarchy and the one rule that secures it	24
10.2 One catalog — modules and administrative actions, identical treatment	24
10.3 Schema	24
10.4 Onboarding — deny-by-default, not opt-out	25
10.5 Feature dependencies — confirm and cascade, both directions	25

10.6 Migrating off the existing privileges array	26
10.7 The runtime check — one function, everywhere	26
10.8 Every dynamic surface, not just navigation	26
10.9 The full lifecycle, end to end	27
10.10 Deliberately deferred — SaaS commercial features, not EMS features	27
11. Organization Setup & Bulk Operations	28
11.1 “Day 0” — new tenant setup	28
11.2 Bulk Onboarding	28
11.3 Bulk Offboarding & the Exit Checklist	29
12. Roles & Responsibilities Matrix	30
13. Data Model Summary	31
13.1 Foundation (shared by both modules)	31
13.2 Leave Management	31
13.3 Payroll	31
14. Phased Implementation Roadmap	33
15. Design Principles — What Ties This Together	34
Appendix A — Restaurant Vertical Extension (Optional)	35
A.1 Prime Cost / Labor-Cost-to-Sales	35
A.2 Tip-Out Distribution Chains	35
A.3 Meal-Period & Covers-Driven Scheduling	35
A.4 Emergency Shift-Swap Broadcast	35
A.5 Station Certification-Gated Scheduling	35
A.6 Split-Shift Premium Automation	35
A.7 Comp / Void / Walkout ↔ Shift Accountability	35
A.8 Staff Meal as a Tracked Benefit	35

1. Executive Summary

Smart Teams began as a single-purpose attendance system: geofenced, face-verified, audit-logged check-in and check-out for distributed teams. This document specifies the next two modules — **Leave Management** and **Payroll** — designed not as bolt-on features, but as a natural continuation of the same architecture that already runs attendance in production.

The guiding principle across both new modules is a single one: **everything that costs the company money, or affects an employee's time-off balance, becomes an immutable ledger entry**. Every dashboard, every report, and every audit question is answered by querying that ledger — not by writing bespoke calculation logic per screen. This is the same discipline the existing attendance module already applies to its hash-chained audit log; Leave and Payroll simply extend it to time-off balances and money.

This document is organized in three layers: **what exists today** (Module 1, for context), **what is being built** (Modules 2 and 3 — every feature and every workflow, including how a real HR/payroll department would actually run this month to month), and **how it all connects** (the shared ledger, policy versioning, and role-based access control that make the three modules behave as one platform instead of three separate apps).

Layer	Contents
Module 1 — Attendance (existing)	Multi-tenant RBAC, GPS/WiFi/Face/QR verification, breaks, corrections, WFH, holidays, audit ledger
Module 2 — Leave (new)	Leave types, versioned policies, an append-only leave ledger, accrual, approval, comp-off, encashment
Module 3 — Payroll (new)	Salary structures, statutory compliance, payroll runs, the financial ledger, loans, reimbursements, cost centers
Integration layer	One shared ledger philosophy, one policy-versioning mechanism, one RBAC system, one scheduler, one systematic notification engine (in-app + email, every event)

2. System Overview — Three Modules, One Platform

Smart Teams is not three separate applications sharing a login page. It is one tenant-scoped platform where each module produces data the next module consumes — the same way a real company's HR and finance departments depend on each other's output.

2.1 The dependency chain

How data flows through the platform

MODULE 1 – ATTENDANCE

Daily check-in/check-out, GPS + WiFi + Face + QR verification, breaks, corrections

|

v (attendance % , unexcused absences, holiday/weekend work)

MODULE 2 – LEAVE MANAGEMENT

Leave balance ledger, requests, approvals, comp-off, encashment

|

v (Loss-of-Pay days, encashable balance, comp-off credits)

MODULE 3 – PAYROLL

Salary structures, statutory deductions, payroll runs, loans, reimbursements

|

v (every earning / deduction / employer cost, as a row)

THE FINANCIAL LEDGER

One immutable, append-only table – the single source of truth for money

|

v

ANALYTICS & DASHBOARDS

Drill-down cost reports: company -> department -> employee -> transaction

Nothing above is a separate database or a separate service. It is one Postgres schema, one Express backend, one React frontend — the same monorepo that already runs attendance. Leave and Payroll are new tables and new routes inside that same system, reusing its authentication, its role/privilege model, its background scheduler, and its notification/email pipeline.

2.2 What makes this different from a typical student/college EMS

- **Nothing is ever silently overwritten.** Leave balances, salary changes, and policy rates are all versioned or ledgered — history is always reconstructable exactly as it was at the time.
- **Every policy has an effective date.** A PF-rate change in October never alters what March's payslip should have paid.
- **Every dashboard number is explainable.** “Why is this employee's PF ■1,800?” has a literal answer stored alongside the number, not just a total.
- **Attendance, Leave, and Payroll speak the same language.** A Loss-of-Pay day computed in Leave is the exact same number Payroll reads — there is no second calculation that could silently drift out of sync.

3. Module 1 Recap — Attendance (Existing, In Production)

This section is intentionally brief — Module 1 is already built and running. It is included so the Leave and Payroll sections that follow can reference exactly what already exists rather than assuming it.

3.1 Organization & access model

- **Multi-tenant** — every company is a tenant; every table is tenant-scoped.
- **Roles:** `super_admin` (platform owner) → `tenant_admin` (company owner) → `manager / HR / GM` → `employee`.
- **Privilege system** — beyond role, individual users can be granted specific permissions (e.g. `employee.create`, `attendance.approve`) checked via a single `hasPrivilege()` function. Leave and Payroll extend this exact mechanism with new privilege strings — no new access-control system.

3.2 Attendance verification

- **GPS geofencing** — check-in validated against the tenant's registered office location and radius.
- **WiFi/IP verification** — optional check against the office's public IP.
- **Face verification** — a separate Python microservice (InsightFace/ArcFace) performs liveness detection and identity matching; the Node app only ever handles the resulting score and embedding, never the model itself.
- **QR attendance** — a rotating, tenant-scoped QR code as an alternative check-in path.
- **Work From Home** — an additive attendance mode with its own eligibility policy (allowed roles, weekdays, monthly cap) and registered home-location geofencing.

3.3 Supporting features already in place

- Break sessions with a daily time budget and geofence-aware exit/entry tracking.
- Attendance corrections — an approval-gated workflow, never a silent edit to a log.
- Tenant-configurable holidays and weekend days, used throughout attendance-percentage and (from Module 2 onward) leave-accrual calculations.
- A hash-chained audit ledger for attendance actions — each entry references the previous entry's hash, making tampering detectable. The Financial Ledger in Module 3 is the same idea applied to money.

What Module 1 does not yet have — and what Module 2/3 introduce for the first time: `department/designation/manager` fields on employees, a real (non-mock) policy version history, tenant-level timezone and currency, and bank account details. These gaps are filled in Phase 0, described in Section 8.

4. Module 2 — Leave Management: Features

4.1 Leave Types & Policies

Every tenant defines its own catalog of leave types (Casual, Sick, Earned, Maternity, Comp-Off, etc.), each marked Paid or Unpaid. The rules governing how each type behaves — annual entitlement, carry-forward cap, whether it's encashable, whether unpaid leave beyond balance is allowed — live in a **versioned policy**, not as live-mutable settings. A policy change never rewrites how last year's leave was calculated.

4.2 The Leave Ledger — append-only balance tracking

Leave balance is never a single mutable number. It is the sum of an append-only ledger of credits and debits, exactly the way a bank balance is the sum of transactions, not a field someone edits directly.

Example — one employee's Earned Leave ledger

```
+12.0 Annual Credit (accrual, Jan)
-2.0   Leave Taken           (usage, Mar 14-15)
-3.0   Leave Taken           (usage, Jun 2-4)
+1.0 Manual Adjustment (HR correction, reason logged)
+5.0 Carry Forward (year-end rollover)
-1.0   Expired               (unused balance past cap)
-----
12.0 = CURRENT BALANCE (sum of every row above)
```

4.3 Accrual

- Monthly automated credit, run by the same background scheduler that already powers attendance's daily jobs.
- New joiners are prorated from their join date in their first accrual cycle rather than receiving a full year's credit on day one.

4.4 Leave Requests

- Apply, cancel, and view balance/history — full self-service, no HR ticket needed for the common case.
- **Half-day leave** — morning/afternoon, deducts 0.5 days.
- **Sandwich leave** — an intervening weekend/holiday between two leave dates can optionally count against balance too (tenant policy toggle), reusing the exact working-day-exclusion logic already written for attendance-percentage calculations.
- **Single-level approval** — reuses the existing privilege-gated approval pattern already used for attendance corrections and WFH location changes; no new workflow engine.

4.5 Comp-Off

Comp-off lifecycle

```
Employee works a day flagged Holiday/Weekend in attendance
      |
v
System marks the day ELIGIBLE for a comp-off claim
      |
v
Employee submits a CLAIM (not automatic — avoids false positives
such as a casual login on a day off)
      |
v
```

Manager APPROVES ----> Leave Ledger credited
 |
 v (approval expiry, if unclaimed for N days)
 Credit auto-expires if unused after the policy's configured window

4.6 Restricted / Floating Holidays

A region's government-recognized holidays are rarely all mandatory for every employee — different festivals matter to different communities. This is the standard “restricted holiday” pattern: a pool of optional holiday dates, from which each employee picks only as many as their entitlement allows, and that entitlement is never locked to a specific festival — unused credit can be spent against any other date in the pool.

The pool — extending the existing holidays table

The existing holidays table (currently just `id`, `tenantId`, `date`, `name`) gains four columns:

Column	Purpose
type	'mandatory' (applies to everyone) or 'restricted' (part of the optional pool)
region	State/city; null = tenant-wide. A Tamil Nadu office and a Delhi office don't share the same regional festivals
blockGroup	Groups a multi-day festival together ('diwali-2026') so the UI can show “Diwali (3 days available)” as one labeled group
source	'manual' or 'calendar_sync' — see the sync note below

Entitlement, picking, and the redirect behaviour

Restricted Holiday is simply a new `leave_type` — paid, with an annual entitlement (e.g. 2–3 days). Picking a pool date debits the `leave_ledger` exactly like any other leave usage. Because the entitlement is “N days, spendable against any pool date” rather than “N Diwali days,” the redirect/banking behaviour falls out for free: an employee who skips Diwali entirely and spends the same credit extending Christmas has done nothing special — the pool model already permits it, with no separate swap transaction required.

- Picks are **auto-confirmed**, not routed through manager approval — the tenant already sanctioned the whole pool in advance, so approval adds friction with no real gatekeeping value here.
- Unused credit expires if not spent within a configured window (e.g. 6 months) — the same expiry mechanic already built for comp-off, reused rather than reinvented.
- `GET /api/leave/holiday-pool?month=X` joins pool dates (filtered by the employee's region) against their remaining balance, answering “how many can I take this month” directly.

Keeping the calendar current

“Updated from the calendar” needs an honest, specific answer rather than an assumed live integration: v1 is a yearly CSV a tenant admin uploads per region (source: `'manual'`), with a public holiday API as a later, pluggable source: `'calendar_sync'` option once one is actually selected and verified for the regions that matter. Region tagging works today with a plain `region` text field on both `holidays` and `users` — it does not require the full `Branch/Organization-Structure` entity that remains deferred elsewhere in this document.

4.7 Leave Encashment

For leave types marked encashable, unused balance can be converted to money. This is not a standalone feature — it is one additional input to Module 3's payroll-run generation, computed from the same leave ledger and written straight into the Financial Ledger (Section 6) as an earning.

4.8 Loss of Pay (LOP) — the join to Payroll

A single function, `getLopDaysForPeriod(employee, month, year)`, combines two sources: leave requests that exceeded available balance (when the tenant's policy allows unpaid leave), and unexcused absences already visible in attendance logs with no covering leave request or correction. This function is **the** literal integration point between Attendance, Leave, and Payroll — built once in Module 2, consumed by Module 3.

5. Module 2 — Leave Management: Workflows

5.1 Employee workflow

Applying for leave

1. Employee opens Leave tab -> sees current balance per leave type
2. Selects dates, leave type, half-day/full-day, attaches a reason (medical certificate required automatically if Sick Leave > 3 consecutive days)
3. System checks available balance in real time
 - Sufficient balance -> request submitted as PENDING
 - Insufficient, policy allows -> excess days flagged for LOP, still submitted
 - Insufficient, policy blocks -> rejected immediately with a clear reason
4. Employee can CANCEL a pending or future-dated approved request at any time -> cancellation writes a compensating credit, nothing is deleted

5.2 Approver workflow

Reviewing a request

1. Manager/HR (whoever holds 'leave.approve') sees pending requests for their team
 2. Reviews dates against team calendar, attendance history, balance
 3. APPROVE -> Leave Ledger debited, employee notified by email + in-app
- REJECT -> reason recorded, employee notified, balance untouched

5.3 Automated / system workflow

Job	Cadence	What it does
Leave accrual	Monthly	Credits each active employee per their policy's entitlement
Comp-off expiry	Daily	Expires unused comp-off credits past the policy window
Carry-forward / reset	Yearly (year-end)	Rolls unused balance forward up to the cap, or resets/flags for encashment

5.4 The professional workflow — how a real HR team actually runs this

This is the part that matters most for feeling like real company software rather than a form-submission demo. A functioning leave system isn't just "apply / approve" — it has a rhythm across the employee lifecycle and the calendar year.

New joiner

Onboarding

HR creates the employee record -> assigns a leave policy
 | (department / role determines which policy applies automatically)
 v
 First accrual cycle is PRORATED from the join date, not a full year's credit
 |
 v
 Employee sees an accurate starting balance from day one -- no manual HR spreadsheet entry

The ongoing monthly rhythm

A typical month

Throughout the month - employees apply, managers approve, balance ledger updates live

Month-end - HR reviews any unusual patterns (e.g. repeated single-day Sick Leave right after paydays -- a known abuse pattern in real payroll departments)

Month-end - LOP days for the period are now fixed inputs, ready for Payroll to consume (see Section 7.1 for the combined monthly close)

Employee exit

Resignation / termination

Final working day recorded

|

v

Leave Ledger balance frozen -> encashable portion computed per policy

|

v

Encashment amount flows into Final Settlement in Payroll (Section 7.4)

Year-end

- Unused balance either carries forward (up to the policy cap) or is auto-encashed, per each leave type's configured rule.
- A fresh policy version can be published for the new year (e.g. revised entitlement) without touching any historical record — old requests remain tied to the policy version that was active when they were approved.

6. Module 3 — Payroll: Features

6.1 Salary Structures & Components

Each employee's compensation is a **versioned** salary structure (never overwritten — a revision creates a new version, effective from that date), broken into normalized components rather than one JSON blob. This is deliberate: Finance frequently needs to ask “show everyone receiving Fuel Allowance,” which is a plain filtered query against a component table and an awkward one against a JSON array.

6.2 Payroll Component Catalog

Every allowance, deduction, and employer cost — Basic, HRA, PF, Fuel, Insurance, Training budget, and any custom ones a tenant adds — is defined once in a shared catalog, not hardcoded. Each component carries:

- **Category** — Earning / Deduction / Employer Cost / Reimbursement
- **Component Group** — Salary / Allowance / Benefit / Tax / Contribution / Reimbursement / Loan / Bonus (how Finance likes to group a P&L)
- **Formula type** — Flat amount, Percentage of Basic, or Percentage of Gross (deliberately not a full formula language — bounded and predictable)
- **Visibility flags** — Visible on Payslip / Visible to Employee / Included in CTC / Taxable. A laptop-depreciation employer cost, for example, belongs in CTC reporting but must never appear on a payslip — these flags are how that distinction is enforced.

6.3 Statutory Compliance (India)

PF, ESI, Professional Tax, and TDS rates live in a **versioned policy**, exactly like leave policies — a rate change takes effect from a date, and never silently recalculates a prior period. The Loss-of-Pay proration method (actual days in the month vs. a fixed 30-day basis) is an explicit per-tenant setting, since the two methods produce different deductions in a 28-day February versus a 31-day month — a common source of quiet payroll bugs when left implicit.

6.4 The Payroll Run — a controlled processing unit

Payroll is never “generate and done.” It is a stateful record with its own lifecycle, so every run can answer who processed it, when, and whether it can still be changed.

Payroll Run status

DRAFT -> APPROVED -> LOCKED -> POSTED -> ARCHIVED

DRAFT numbers computed, nothing final, freely re-generatable
 APPROVED reviewed and signed off, ledger rows move draft -> approved
 LOCKED no further automatic recompute, even if attendance is corrected later
 POSTED final -- ledger rows count in every dashboard, payslips generated, emailed
 ARCHIVED terminal state, retained for compliance

6.5 The Financial Ledger — single source of truth

Every component of every payroll run — Basic, HRA, PF (employee and employer separately), LOP, a reimbursement, a loan EMI, an encashment payout — becomes one row. Nothing is computed live inside a dashboard; dashboards only aggregate what is already here.

Column	Purpose
componentCode / componentGroup	What this row is — Basic, PF, Fuel — and which broad group it rolls up into

Column	Purpose
category	Earning / Deduction / Employer Cost / Reimbursement
amount, currency	Currency copied from the tenant at write-time, so it never silently changes retroactively
status	draft / approved / posted / reversed — real accounting never deletes, it reverses
metadata (JSON)	The calculation trace — formula, inputs, output — so any number is explainable
payrollRunId, costCenterId	Which run produced this row, and which project/cost center it belongs to
department / designation / role snapshot	Captured at write-time, so a department cost report from last year stays correct even after someone transfers teams
prevHash, hash	Hash-chained, the same tamper-evidence pattern as the existing attendance audit ledger

6.6 Loans & Salary Advances

One table, two use cases: a salary advance is simply a loan with a one-month tenure. `loanType` distinguishes Salary Advance / Personal / Emergency / Education / Relocation. Once approved, the payroll run automatically deducts the EMI each month and decrements the outstanding balance until it reaches zero.

6.7 Reimbursements

Employees submit a claim with a receipt (category, amount, description, upload) against the same component catalog used everywhere else. States: Draft → Submitted → Approved / Rejected → Included in Payroll → Paid, plus Cancelled. Once approved, the next payroll run picks it up automatically and writes the ledger entry — no separate manual payment step.

6.8 Cost Centers

An employee can be allocated across multiple cost centers (e.g. 60% Project A, 40% Project B), versioned with effective dates since allocations change over time. When active, the payroll run splits each component's ledger write proportionally, so “how much did Project X cost us” is answerable directly from the ledger.

6.9 Bank Accounts & Payslips

- Bank details live in their own versioned table (account number, IFSC, holder name), never as a live-mutable field on the employee record — a payroll run snapshots which account was used, so historical payslips remain accurate even after an employee changes banks.
- Payslips store the full breakdown (gross, total employee deductions, total employer contributions, net pay, total employer cost / CTC) as real columns, generated once per employee per posted run, emailed automatically.

6.10 Payroll Validation Report

Before any run can be posted, a validation pass runs automatically and produces a structured report — not a silent pass/fail.

Example validation report

```
198 Employees in this run
196 Valid
2 Missing bank details <- BLOCKS posting for these employees
1 Negative net salary <- BLOCKS posting for this run
```

```
4 Pending attendance corrections (warning, does not block)
3 Pending leave requests overlapping this period (warning)
1 Salary structure missing <- BLOCKS posting for this employee
```

6.11 Payroll Simulation

Because the payroll engine separates *computing* the numbers from *persisting* them, HR can simulate a policy change — “what if PF goes from 12% to 13%?” — and see the impact (total cost before/after, difference, number of employees affected) with zero data written. Same computation function, run twice, diffed.

6.12 Multi-Approval, Accounting/Bank Export, Budgets

- **Multi-approval** — a configurable count of distinct approvals required before a run can lock, without hardcoding specific role names into the schema.
- **Accounting export** — posted ledger entries, grouped by component group, mapped to a tenant's own chart of accounts, exported as a debit/credit journal. Built pluggable (one format now, e.g. generic CSV; additional formats such as Tally added only on real customer demand).
- **Bank export** — a NEFT-style bulk-upload file generated from posted payslips + bank account snapshots. Same pluggable-format principle.
- **Payroll budgets** — a per-department, per-period budget figure compared against actual posted spend, powering a budget-vs-actual dashboard.

7. Module 3 — Payroll: Workflows

This section is the heart of “how a company professional workflow would be” — the monthly payroll close is a real, well-established business process at any company, and the system is designed to mirror it exactly rather than reduce it to a single “Generate Payroll” button.

7.1 The monthly payroll close — a full-month timeline

This is how an HR/Finance team actually runs a month, from the first working day through bank transfer and accounting close. Dates below are illustrative for a typical Indian monthly-payroll company; the actual cutoffs are tenant-configurable.

When	What happens	Who
Day 1 - 25	Employees clock in/out; leave is applied and approved; attendance corrections are requested and approved as needed	All employees, Managers
~Day 25	Attendance correction window closes for the period. Late corrections roll into next month's retroactive adjustment, not this month's run	HR
~Day 26	Payroll run created in DRAFT. System pulls: current salary structures, active statutory policy, LOP days (Section 4.7), approved reimbursements, active loan EMIs, any encashment due this cycle	HR / Payroll admin
Day 26 - 27	Validation Report reviewed (Section 6.10). Any hard-blocking issues (missing bank details, negative pay, missing salary structure) resolved before proceeding	HR
Day 27 - 28	Numbers reviewed via drill-down — total cost, department cost, individual anomalies (a duplicate component, an unusually large jump vs. last month) checked against the ledger's calculation metadata	HR, Finance
Day 28	Run APPROVED. Required sign-off count satisfied. Ledger rows move draft -> approved	Whoever holds payroll.approve
Day 29	Run LOCKED. No further automatic recompute is possible from this point — any further attendance correction becomes a retroactive adjustment landing in *next* month's run, never edits this one	HR
Day 30 / 1st	Run POSTED. Ledger rows become final and count in every dashboard. Payslips generated and emailed. Bank export file generated and uploaded to the bank portal for NEFT transfer	Finance
1st - 3rd	Accounting export generated for the finance team's books; PF/ESI/PT/TDS statutory filings prepared from the same posted ledger	Finance

7.2 Employee-facing workflow

- View current CTC breakdown (gross, deductions, employer cost) at any time — not just on payday.
- Download payslips and view payment history once posted.
- Submit reimbursement claims with receipts; track claim status through to payment.
- Request a salary advance or loan; track EMI deductions against outstanding balance.
- See exactly why a number is what it is — clicking a payslip line shows the same calculation trace HR sees (Section 6.5's ledger metadata), just scoped to that employee.

7.3 HR / Finance workflow

This is the Section 7.1 timeline from the operator's seat — the actions available at each stage of a run's lifecycle:

Actions available per Payroll Run status

DRAFT -> re-generate (if data changed), review drill-down, run validation
 APPROVED -> lock, or revert to draft if an issue is found before locking
 LOCKED -> post, or (rare) an authorized unlock with reason logged to the audit trail
 POSTED -> generate exports (bank file, accounting journal), view-only from here

7.4 New joiner → Payroll

Bringing a new employee into payroll

HR creates the employee record, assigns department/designation/manager
 |
 v
 HR creates the employee's first Salary Structure (effective from join date)
 |
 v
 Employee optionally submits bank account details (self-service, same optional-until-completed pattern as KYC)
 |
 v
 First payroll run after joining automatically prorates pay for a partial month

7.5 Employee exit → Final Settlement

Resignation / termination

Last working day confirmed
 |
 v
 System computes final settlement:
 + Pending salary up to last working day (prorated)
 + Leave encashment (unused balance x per-day rate, per policy)
 - Outstanding loan balance (recovered in full, or per exit policy)
 + Any pending approved reimbursements
 - Notice-period shortfall, if applicable
 |
 v
 Final settlement run generated -> same validation, approval, posting pipeline as a normal run, just flagged as a one-time final settlement for that employee

7.6 Year-end

- Leave carry-forward/expiry runs (Section 5.3) feed encashment amounts into the appropriate payroll run.
- Updated statutory rates for the new financial year are published as a new policy version, effective from a specific date — every payslip generated afterward automatically uses the new rate; every payslip already posted stays exactly as it was.
- Annual cost reports and department-level drill-downs (Section 6.5's Financial Ledger, aggregated over a full year) become available once a full year of posted ledger data exists.

7.7 Payroll Correction / Retro-Pay

What actually happens when a **posted** payroll run turns out to have an error — answered directly rather than left implicit in the run-lifecycle description:

An error is found in a run that's already posted

Error discovered (wrong attendance correction applied late, a missed reimbursement,

```
a salary structure that should have been effective a month earlier)
|
v
The POSTED run is never edited -- its ledger rows stay exactly as they were,
status stays 'posted' (Section 6.5's reverse-not-delete principle)
|
v
A retroactive_adjustment ledger entry is created instead:
  - references the ORIGINAL payrollRunId in its metadata
  - amount is the DELTA only (what should have been paid, minus what was),
not a restatement of the full figure
  - status: 'draft', same approval gate as any other ledger write
|
v
Approved through the SAME payroll.approve privilege as a normal run --
no separate 'correction approver' role invented
|
v
The delta lands in the NEXT payroll run's net pay, clearly labelled as an adjustment
on that payslip, with a reference back to which month it corrects
```

This is not a new mechanism — it's the retroactive-adjustment behaviour already specified for attendance-correction-after-lock (Section 6.4) generalized to any post-posting error, using the same ledger, the same approval privilege, and the same audit trail.

8. Cross-Module Integration — The Foundation Layer

Every feature above depends on a small set of shared building blocks, added once and reused by both new modules rather than duplicated. This is what makes the platform coherent instead of three modules that happen to share a login.

8.1 Tenant-level timezone & currency

Two new fields on the existing tenant record. Since each tenant is a distinct company, different clients can genuinely be in different countries — this is not a foreign-exchange conversion engine, just what a tenant's numbers are denominated and displayed in, and which clock its day/week/month boundaries are computed against. This also fixes a latent bug: date-boundary logic (which weekday is “today,” month-boundary attendance calculations) previously used the server's local clock rather than the tenant's.

8.2 Real policy versioning

A single, reusable mechanism: every policy (leave rules, statutory rates) is a dated, immutable version, not a live-mutable setting. This did not exist anywhere in the platform before — it is introduced once, generically enough that attendance policies could adopt the same mechanism later if desired, without needing a second, differently-built version system.

8.3 Role & privilege extension

No new access-control mechanism — new privilege strings slot into the exact `hasPrivilege()` system already gating attendance actions:

New privilege	Grants
leave.approve	Approve/reject leave requests and comp-off claims
leave.manage	Configure leave types and publish leave policy versions
payroll.run	Generate, review, and approve payroll runs
payroll.view	View payroll reports and dashboards without run authority
reimbursement.approve	Approve/reject reimbursement claims
loan.approve	Approve loan and salary advance requests

8.4 Scheduler — organized by cadence

Cadence	Jobs
Daily	Comp-off expiry check, end-of-day attendance auto-checkout (existing)
Monthly	Leave accrual, payroll run auto-draft reminder
Yearly	Leave carry-forward / reset, statutory-rate policy renewal prompt

8.5 The two ledgers, side by side

	Leave Ledger	Financial Ledger
Tracks	Days of leave balance	Money — earnings, deductions, employer costs
Written by	Accrual cron, leave approval, comp-off, encashment trigger	Payroll run generation, reimbursements, loans

	Leave Ledger	Financial Ledger
Never does	Store a mutable "balance" field	Delete a row — always reverses instead
Read by	Leave balance screens, LOP calculation	Every payroll dashboard, cost report, drill-down

Both ledgers exist for the same reason the existing attendance audit ledger exists: a number is only trustworthy if you can show exactly how it was produced. Future modules (assets, training budgets, recruitment costs) are expected to write into the same Financial Ledger rather than invent a fourth ledger of their own.

9. Notification Engine — Systematic, Every Time

Today, notifications are an afterthought: the `notifications` table exists but is written to from a single place in the entire backend, there is no in-app notification UI anywhere in the frontend, and email is sent through twelve separate hand-written functions in `mail.ts` — one per event type. Every new event has meant writing a new function from scratch. That does not scale to two more modules' worth of events, and it means notification coverage depends on a developer remembering to add it per route rather than the system guaranteeing it.

The fix is the same discipline applied everywhere else in this document: one shared mechanism, reused by every module, instead of a bespoke implementation per feature.

9.1 One dispatch function, both channels, always together

The systematic pattern – replaces 12 bespoke email functions

```
notify({
  tenantId,
  recipients: [userIds] OR a privilege string (e.g. 'leave.approve')
    -- resolved via the EXISTING getUsersWithPrivilege() pattern,
  no new recipient-resolution logic
  type: 'leave.approved' -- one short, namespaced event key
  data: { employeeName, dates, ... } -- fills a template for this type
  link: '/leave/requests/42' -- where the notification redirects to
})
```

- |
- v
1. Writes ONE row per recipient to ``notifications`` (in-app)
 2. Renders the subject/body for ``type`` from a shared template registry and sends it through the EXISTING generic `sendEmail()` -- not a new bespoke function per event
 3. Both channels fire from the same call -- never a case where the in-app entry exists but the email didn't, or vice versa

A new Leave or Payroll event becomes one new entry in a template registry (subject + body for that type), not a new exported function. This is what “systematic” means concretely: coverage is a data-table lookup, not something a developer has to remember to wire up correctly on every new route.

9.2 Schema — extending, not replacing, the existing table

Column	Status	Purpose
id, userId, title, message, isRead, createdAt	Existing	Unchanged
tenantId	New	Currently missing — needed for tenant-scoped queries and cleanup
type	New	The namespaced event key (e.g. <code>leave.approved</code> , <code>payroll.posted</code>) driving the template and the icon shown in-app
link	New	The redirect path — what makes the notification center clickable rather than read-only
emailSent	New	Whether the email channel was also dispatched for this row, for auditability

9.3 Every event, by recipient — the complete coverage matrix

This is the systematic part made concrete: every state-changing action across Attendance, Leave, and Payroll that should notify someone, listed once, so coverage can be checked against this table rather than trusted to memory.

Recipient	Notified when...	Redirects to
Employee	Leave request approved / rejected	That leave request's detail view
Employee	Comp-off credited, or expiring soon	Leave balance screen
Employee	Payslip posted and ready	That payslip
Employee	Salary revision becomes effective	Compensation summary
Employee	Loan / advance approved or rejected; each EMI deducted	Loan detail / EMI schedule
Employee	Reimbursement claim approved, rejected, or paid	That claim
Employee	Attendance correction approved / rejected (existing)	That correction
Employee	Bank details missing, payroll cutoff approaching	Bank details form
Manager	A team member has a pending leave request or comp-off claim	Pending approvals queue
Manager	A team member's attendance correction needs review (existing)	Pending corrections queue
HR	An employee is missing a salary structure before a payroll cutoff	That employee's compensation tab
HR	Comp-off credits are expiring unclaimed, tenant-wide	Comp-off report
HR	A payroll run's Validation Report has blocking errors	That run's validation report
HR / Finance	A payroll run is ready for review	That run's drill-down view
Finance	A payroll run is approved and awaiting lock/post	That run
Finance	Bank export or accounting export file is ready	The export download
Finance	A department's spend is approaching or exceeding its budget	Budget dashboard for that department
Tenant Admin	A payroll run has posted — summary of total cost	Payroll run summary

9.4 In-App Notification Center

A bell icon added to the existing dashboard chrome (`PortalShell.tsx` / `BottomNav.tsx`), showing an unread-count badge sourced from `GET /api/notifications/unread-count` — polled the same way the existing QR check-in display already re-fetches on an interval, not a new real-time infrastructure the app doesn't otherwise have.

Clicking a notification – the redirect behaviour

Bell icon (badge: 3)

| click

v

Dropdown/panel: most recent notifications, unread visually distinct
'Leave request from Priya - Pending your approval' 2m ago
'Payslip for April is ready' 1h ago
'Payroll Validation: 2 employees missing bank details' 3h ago
| click one entry

v

1. Marks that notification read (POST /api/notifications/:id/read)
2. Navigates (React Router) directly to its `link`
-- e.g. the payslip notification opens that exact payslip,
not a generic 'Payroll' landing page

- GET /api/notifications — paginated, most recent first
- GET /api/notifications/unread-count — powers the badge
- POST /api/notifications/:id/read and /read-all
- All four reuse the existing authenticate middleware — no new auth mechanism.

9.5 Migration path for the twelve existing email functions

The existing attendance-related functions in `mail.ts` (`sendWfhDecisionEmail`, `sendAttendanceCorrectionEmail`, and the rest) keep working unchanged — nothing breaks on day one. Every **new** Leave/Payroll event is built through `notify()` from the start. The existing functions are folded into the same template registry opportunistically, whenever that specific email is next touched for another reason — not as a risky big-bang rewrite of working code.

10. Feature Entitlement Cascade

This is the mechanism that governs whether any capability described elsewhere in this document — a business module (Restaurant, Prime Cost), a Leave/Payroll feature, or an administrative action (hiring, granting privileges) — is even visible to a given user. Nothing in this platform should be hardcoded to a role string. Every capability, including basic administrative ones, is a toggleable entitlement, delegated downward through a hierarchy where no level can grant more than it itself holds.

10.1 The hierarchy and the one rule that secures it

Super Admin -> Tenant Admin -> GM -> Manager -> Employee

Super Admin

defines the platform's full feature catalog, enables/disables features globally

|

v

Tenant Admin

receives an approved subset at onboarding, delegates any subset of it to GMs

|

v

GM -> Manager -> Employee

same rule repeats at every level, no special-casing per role

THE INVARIANT:

No user can grant a feature or action they do not currently possess.

That single rule secures the entire delegation chain, at every level, forever.

Note what this hierarchy is *not*: it is not an organizational reporting chain. “GM creates Manager” is an administrative action gated by an employee.create entitlement — it does not imply the Manager reports to that GM for HR purposes. Who approves whose leave and who can see whose attendance is a separate concern (managerId/departmentId, deferred org-structure work) and is deliberately not conflated with this delegation mechanism.

10.2 One catalog — modules and administrative actions, identical treatment

There is no separate “admin permissions” system running alongside a “feature flags” system. Everything is a row in the same catalog:

type	Example code	Meaning
module	attendance, leave, payroll, restaurant, prime_cost	A whole capability area is available
action	employee.create, privilege.assign, role.edit, salary.run, shift.assign	A specific administrative or operational action is permitted

The type column exists purely so the toggle-picker UI can group “Modules” separately from “Administrative Actions” — it changes nothing about how entitlement is checked or delegated. Hiring an employee, assigning a privilege, and turning on the Restaurant module are, to this system, the exact same kind of thing.

10.3 Schema

Table	Purpose
features	The platform catalog: code, name, type (module/action), category, isGloballyEnabled
tenant_features	tenantId, featureId, approvedBy, enabledAt, disabledAt — which catalog features a tenant is approved for, with full enable/disable history (same pattern as user_features below)

Table	Purpose
user_features	userId, featureId, grantedBy, grantedAt, revokedAt — who has what, and the full audit trail (never delete a row; revoke by timestamp, insert fresh on re-grant)
feature_dependencies	featureId, dependsOnFeatureId — a bounded dependency graph, not a rules engine

tenant_features is not a single mutable boolean — every enable/disable is a new row (or a stamped disabledAt on the current one, with a fresh row on re-enable), so a question like “why wasn’t payroll running in February” has a direct, queryable answer: the Restaurant module (which Payroll depended on) was disabled Feb 10 and re-enabled Mar 5. This is the same discipline as everything else in this document — never overwrite state that has historical value.

user_features is deliberately normalized, not a JSON array on the user record — easier revocation, easier “who has X” queries, and the grantedBy/grantedAt/revokedAt columns are the audit trail on their own, no separate log table required. The effective feature list for a session is computed once at login and cached (JWT or server-side session) so the hot runtime check stays O(1) regardless of how normalized the underlying storage is.

10.4 Onboarding — deny-by-default, not opt-out

Tenant onboarding flow

```
Tenant Admin signs up
    |
v
Selects the features they want from the platform's full catalog
    |
v
Super Admin reviews the request -> approves fully, or prunes specific items
    |
v
Tenant receives exactly the approved set -- nothing is on by default
```

Deny-by-default rather than everything-on-by-default is a deliberate choice: this system gates capabilities like `privilege.assign` and `salary.run`, and relying on Super Admin to notice and disable an inappropriate default is a weaker security posture than an explicit, reviewed allowlist. This manual review flow is also the direct precursor to automatic plan-based provisioning once pricing tiers exist (Basic -> 25 features, Professional -> 60, Enterprise -> all) — same mechanism, the approval step just becomes automatic.

This request-and-approve flow is not onboarding-only. The identical mechanism handles a tenant asking for an additional feature six months in — `POST /api/tenant/feature-requests` at any time, reviewed the same way. There is exactly one request/approve code path, used at signup and reused for every request after it, not a special first-time-only step.

10.5 Feature dependencies — confirm and cascade, both directions

Some features are meaningless without another — Prime Cost Dashboard depends on Payroll data; Restaurant Analytics depends on the Restaurant module being on. `feature_dependencies` makes this explicit and self-describing, rather than a fact every future engineer has to independently rediscover.

Enabling and disabling with dependencies

```
ENABLE Prime Cost Dashboard
  -> depends on: Payroll, Restaurant
  -> UI shows: "Enabling Prime Cost will also enable Payroll and Restaurant" -> confirm

DISABLE Payroll
  -> Prime Cost Dashboard and Salary Run both depend on it
  -> UI shows: "This will also disable Prime Cost Dashboard and Salary Run" -> confirm
```

Both directions are confirm-and-cascade — never a silent auto-enable, and never a hard block with no path forward. As the catalog grows from the current handful of features toward hundreds, this table is what keeps dependency logic in data instead of scattered conditionals across the codebase. One integrity rule the write path enforces: inserting a `feature_dependencies` row that would create a cycle (A depends on B, B depends on A, directly or transitively) is rejected outright — a standard topological-sort check on write, not a new subsystem. Dependencies stay hard-only (“requires”, not “works better with”) until a real feature actually needs the softer distinction — same bounded-scope discipline as the formula types in Section 6.1.

10.6 Migrating off the existing privileges array

The existing `users.privileges` JSON array (Module 1, live in production today) is not exempt from this system — leaving it as a permanent parallel mechanism would break the “one catalog, one rule” principle this whole design exists to establish. The migration is staged, not a rip-and-replace:

Staged migration, verified the same way as every other production change

1. Build `features` / `user_features` / `feature_dependencies` fresh -- zero risk, nothing depends on them yet
2. One-time data migration: seed the catalog with every privilege string already in use; seed `user_features` from every existing user's current privileges + role defaults
3. Rewrite `hasPrivilege()` / `getEffectivePrivileges()` INTERNALS only -- signatures stay identical, so none of the ~74 existing route handlers that call them need to change
4. Verify: typecheck, tests, a smoke script against real Postgres confirming login and privilege-check behaviour is byte-identical before and after
5. Only then retire the `privileges` column

10.7 The runtime check — one function, everywhere

`hasFeature()` -- the only check any route or UI element makes

`hasFeature(user, featureCode):`

1. Is the feature globally enabled? -- Super Admin's kill switch
2. Has the tenant been approved for it? -- Tenant Admin's cascade
3. Does the user hold it (own grant or role default)? -- the existing, now-normalized privilege mechanism

Every screen, every API route, checks only `hasFeature("salary.run")`.
No role checks. No tenant checks. No hardcoding, anywhere.

On the frontend specifically, this replaces the hardcoded role-string checks that exist today (e.g. `canClockIn(role)`, `canSeeDashboard(role)` in `AdminApp.tsx`) with one dynamic list, fetched once at login (`GET /api/me/features`) and checked by membership everywhere nav items, dashboard cards, and routes decide what to render.

10.8 Every dynamic surface, not just navigation

“Nothing hardcoded” is stated as a principle in 10.7 — this is what it means concretely, surface by surface. Each one is a render loop over the same `GET /api/me/features` list, not a separate mechanism per surface:

Surface	Driven by feature membership
Sidebar / navigation	A nav item exists only if its feature code is in the user's list — no route is ever reachable through the UI without also passing <code>hasFeature()</code> on the API side
Dashboard cards / widgets	A card renders only if its underlying feature (and, per Section 10.5, that feature's dependencies) is present

Surface	Driven by feature membership
Reports	A report is itself a feature code — “Payroll Cost Report” doesn't render for a user without payroll.view, same as any module
API surface	Every route handler's first check is hasFeature() — the API is the enforcement point; the UI hiding an element is convenience, not the security boundary

The API check is what actually matters — a hidden sidebar item is a UX nicety, not protection. If `hasFeature()` isn't checked server-side on the route itself, hiding it client-side is cosmetic only.

10.9 The full lifecycle, end to end

From platform definition to a running check, in one picture

Super Admin defines a feature in the catalog, sets `isGloballyEnabled`

|

v

Tenant Admin requests it (onboarding, or any time after -- Section 10.4)

|

v

Super Admin approves (or prunes) the request

|

v

Tenant Admin delegates a subset to a GM

|

v

GM delegates a subset of THEIR set to a Manager (same rule, every level)

|

v

Manager delegates a subset to an Employee

|

v

Every grant/revoke at every step writes a `user_features` / `tenant_features` row

|

v

`hasFeature()` reads the cached, effective result -- sidebar, dashboard, API, all of it

10.10 Deliberately deferred — SaaS commercial features, not EMS features

A distinct category worth naming explicitly, so it isn't mistaken for a gap: feature **versioning** (running Prime Cost v1 and v2 side by side for different tenants), feature **usage analytics** (adoption dashboards for the platform owner), **vertical packs/templates** (bundling a preset like “Restaurant Starter” for faster onboarding), feature **expiry**, and **trial periods**. These are real, standard capabilities of a mature multi-tenant SaaS product — but they are product/commercial concerns (billing, packaging, growth) layered on top of this entitlement system, not requirements of the EMS itself. None of them are needed for Phase 0–5 to work correctly, and building them now would mean designing for a subscription/billing model that doesn't exist yet. They plug into the schema here cleanly whenever that becomes a real, separate requirement — features already has room for a `planRequirement` field, and vertical packs are just a named, pre-selected list of feature codes on top of the existing catalog, not a new mechanism.

11. Organization Setup & Bulk Operations

Three workflows that don't belong specifically to Leave or Payroll, but that every tenant needs early and at scale: standing up a brand-new company on the platform, importing a workforce all at once rather than one employee at a time, and offboarding at the same scale. Each reuses infrastructure already specified elsewhere in this document.

11.1 “Day 0” — new tenant setup

From signup to a working team

```
Tenant Admin signs up -> requests features (Section 10.4)
    |
v
Super Admin approves -> tenant has its entitled feature set
    |
v
Tenant Admin configures:
  - Departments / designations (flat fields on users -- Section 10.1's deferred
org-structure note applies; Branches specifically remain out of scope until
that entity is actually built)
  - Shift / attendance policy (existing Module 1 config)
  - Leave policy -- first policy_version, domain='leave' (Section 4.1)
  - Payroll policy -- statutory rates, domain='payroll' (Section 6.3)
  - Salary structure templates (Section 6.1)
    |
v
Tenant Admin creates GMs/Managers, delegating a subset of the tenant's features
    |
v
First employees invited -- individually, or via Bulk Onboarding (11.2) for scale
```

Nothing here is a new mechanism — this is a guided sequencing of steps that already exist individually elsewhere in this document, presented as one coherent “getting started” path rather than left for a new tenant admin to discover on their own.

11.2 Bulk Onboarding

Importing a workforce, not one employee at a time

```
HR uploads a CSV/Excel file (name, email, role, department, designation, ...)
    |
v
Row-level validation BEFORE any writes:
  - required fields present, email format valid, role is a real role,
no duplicate emails within the file or against existing employees
    |
v
Error report shown -- exactly which rows failed and why, before committing anything
    |
v
HR chooses: fix and re-upload, or proceed with valid rows only
    |
v
Valid rows are created; invited employees get the standard activation email
    |
v
Import summary: N created, M skipped, full list of skipped rows with reasons
```


Partial success by default, not all-or-nothing — a single bad row (a typo'd email, a duplicate) shouldn't block 499 valid ones. Each created employee gets whatever feature defaults the importing user is entitled to delegate (Section 10), same as one-at-a-time creation — bulk import is a faster path through the existing creation flow, not a separate, less-audited one.

11.3 Bulk Offboarding & the Exit Checklist

Extends the individual Final Settlement workflow (Section 7.5) with the surrounding administrative steps, both for one departing employee and for a batch (e.g. a seasonal team's contract ending together):

- **Disable login** — immediate, via the existing session-revocation mechanism (JWT session invalidation already used for password resets).
- **Final Settlement** — runs exactly as specified in Section 7.5.
- **Exit checklist** — a configurable list of steps a manager/HR ticks off (access cards returned, equipment returned, exit interview completed). Kept deliberately lightweight: a checklist of boolean items logged to the audit trail, not a dependency on an Assets Management module that doesn't exist yet — “equipment returned” is a checkbox HR confirms, not a system that tracks serialized asset inventory.
- **Archive, not delete** — the employee record is marked inactive, never removed; attendance/leave/payroll history remains queryable, consistent with the reverse-not-delete principle applied everywhere else in this document.

12. Roles & Responsibilities Matrix

Who can do what across Leave and Payroll, mapped onto role. Read this as the **typical default bundle** a role receives, not a hardcoded gate — per Section 10, the actual mechanism is the feature entitlement cascade, and any of these can be individually granted or withheld per user regardless of role. A checkmark below means “this is what role defaults to,” not “this is the only way it can be.”

Action	Employee	Manager	HR	Tenant Admin
Apply / cancel own leave	Yes	Yes	Yes	-
Approve leave requests	-	Team only	Yes	Yes
Configure leave policy	-	-	Yes (if granted)	Yes
View own payslip / CTC	Yes	Yes	Yes	-
Submit reimbursement claim	Yes	Yes	Yes	-
Approve reimbursement / loan	-	If granted	Yes	Yes
Generate payroll run	-	-	If granted	Yes
Approve / lock / post payroll run	-	-	If granted	Yes
View department cost dashboards	-	Own team	Yes	Yes
Configure statutory policy rates	-	-	-	Yes
Export bank / accounting files	-	-	If granted	Yes

13. Data Model Summary

Every new table introduced across Modules 2 and 3, for quick reference. All are additive — no existing attendance table is modified in a breaking way; only two nullable columns are added to the existing employee record (department, designation, manager reference).

13.1 Foundation (shared by both modules)

Table	Purpose
financial_ledger	The single source of truth for every earning, deduction, and employer cost
policy_versions	Dated, immutable policy versions — leave rules, statutory rates
payroll_runs	One row per payroll cycle; the controlled processing unit (Section 6.4)
cost_centers	Projects/centers an employee's cost can be allocated against
notifications (extended)	In-app + email dispatch for every event across all three modules (Section 9)
features	Platform-wide feature/action catalog, Super Admin managed (Section 10)
tenant_features	Which catalog features a tenant has been approved for (Section 10)
user_features	Normalized per-user grants with a built-in audit trail (Section 10)
feature_dependencies	Which features require which others (Section 10)

13.2 Leave Management

Table	Purpose
leave_types	Tenant's catalog of leave categories
leave_ledger	Append-only credit/debit history — balance is its sum
leave_requests	Applications for leave, with approval status and computed LOP days
comp_off_claims	Eligibility -> claim -> approval trail for compensatory off
holidays (extended)	Existing table +type/region/blockGroup/source — mandatory vs. restricted-pool holidays (Section 4.6)

13.3 Payroll

Table	Purpose
salary_structures / salary_structure_components	Versioned per-employee compensation, normalized into rows
payroll_components	Catalog of every allowance/deduction/employer cost, with visibility flags
employee_bank_accounts	Versioned bank details, snapshotted per payroll run
payslips	Per-employee, per-run summary — gross, deductions, employer cost, net
loans	Loans and salary advances (one table, distinguished by loanType)
reimbursement_claims	Employee expense claims through to payment

Table	Purpose
employee_cost_center_allocations	Versioned % split of an employee's cost across projects
accounting_gl_mappings	Tenant's chart-of-accounts mapping for journal export
payroll_budgets	Budget figures compared against actual posted spend
payroll_run_approvals	Sign-off records when multi-approval is configured

14. Phased Implementation Roadmap

The build order that gets to a working, demoable system fastest — each phase leaves the application fully functional, nothing is ever half-built in production.

Phase	Delivers
0 — Foundation	Financial Ledger, Policy Versions, tenant timezone/currency, department/designation/manager fields, and the Feature Entitlement Cascade (Section 10) — gating everything built in every phase after this one. No user-facing UI yet.
1 — Leave Management	Leave Ledger, types, requests, accrual, approval, sandwich/half-day leave, Restricted/Floating Holidays, the LOP calculation function
2 — Payroll	Salary structures, statutory calculation, the full run lifecycle, payslips
3 — Dashboards & Analytics	Drill-down cost reporting, plus an entitlement Audit Dashboard (who granted/revoked which feature, when — a UI screen over the Section 10.3 grant history, no new backend) — both mostly free once the ledger and entitlement tables exist, near-zero new schema
4 — Extended Finance Features	Comp-off, encashment, loans/advances, reimbursements, cost-center allocation, bank/accounting export, validation report, simulation, multi-approval, budgets
5 — Vertical Extensions (optional)	Restaurant-specific features (Appendix A) — built only if this deployment is actually a restaurant, gated by the same entitlement cascade as everything else

Feature Entitlement moving into Phase 0 is deliberate, not incidental: gating a capability after it's already built means retrofitting every route and UI element after the fact; gating it from the start means every module built in Phases 1 through 5 is born behind a toggle. Section 10's staged migration off the existing privileges array (10.6) should also happen during Phase 0, while the least code depends on the old shape.

Deliberately out of scope until a real requirement demands them: multi-country taxation, union/collective-bargaining agreements, stock grants/ESOPs, and AI-flavored forecasting or fraud-detection features. All deterministic validation (negative pay, missing bank details, duplicate components) is built as real rule-based checks, not machine learning — the distinction matters, since these are reliable, auditable, testable checks rather than a black box.

15. Design Principles — What Ties This Together

- **One ledger, many views.** Every dashboard is a filtered aggregate query over the Financial Ledger — no bespoke calculation logic duplicated per screen.
- **Reverse, never delete.** Corrections, cancellations, and adjustments write new rows; nothing is silently edited or removed. This is true of attendance corrections today, and of every new leave/payroll record.
- **Snapshot at the moment of truth.** Department, designation, salary, and bank account are captured at the time a transaction happens, so historical reports remain accurate even after an employee transfers teams, gets a raise, or changes banks.
- **Policies are dated, not live settings.** A rate or rule change takes effect from a date and is retrievable exactly as it was for any historical calculation.
- **Reuse the access-control and scheduling systems that already exist.** No second RBAC system, no second background-job runner — new privilege strings and new scheduled jobs slot into what Module 1 already built and proved in production.
- **Bounded, not speculative.** Formula types are Flat / % of Basic / % of Gross, not a general expression language. Export formats are pluggable but only one is built until a real customer needs another. The biggest risk to a system like this is over-engineering for hypothetical requirements instead of the one in front of you.

Appendix A — Restaurant Vertical Extension (Optional)

Everything in this appendix is **optional and gated by the Feature Entitlement Cascade (Section 10)** — built only for deployments that are actually restaurants, invisible to every other tenant. None of it is part of the core Phase 0–4 roadmap; it is Phase 5, pursued only if there is a real restaurant customer to build it for. Every idea below extends infrastructure already specified elsewhere in this document rather than introducing a second architecture.

A.1 Prime Cost / Labor-Cost-to-Sales

The one metric a restaurant owner or GM checks obsessively, and the headline reason this vertical is worth pursuing at all. One small table — `daily_sales` (`tenantId`, `date`, `mealPeriod`, `salesAmount`, `coversCount`) — manager-entered, no POS integration required for v1 — makes labor cost % of sales a live aggregate query over the Financial Ledger that already exists, broken down by meal period, station, or shift exactly like every other ledger-backed dashboard in this document.

A.2 Tip-Out Distribution Chains

Restaurants run codified tip-out chains, not simple pooling — a server pays a fixed % of their tips to the bar, a % to bussers, sometimes a % to kitchen. Modeled as a new `payroll_components` formula type (`tip_out_chain`), computed from each server's shift totals, writing one Financial Ledger row to every recipient — the same component-and-ledger pattern already used for every other payroll component, just modeling a chain instead of a flat split.

A.3 Meal-Period & Covers-Driven Scheduling

Staffing follows brunch/lunch/dinner/late-night demand, using historical covers plus today's reservation count rather than generic occupancy — the restaurant-specific version of demand-aware rostering.

A.4 Emergency Shift-Swap Broadcast

A no-show 30 minutes before dinner service broadcasts an urgent cover request to every off-duty employee certified for that station — first to accept gets it, manager confirms. One new event type on the existing Notification Engine (Section 9), race-to-accept instead of passive read.

A.5 Station Certification-Gated Scheduling

Grill, sauté, pastry, bar, and wine service are internal skill badges, gated the same way as statutory certifications (food handler, alcohol service) — an employee cannot be scheduled into a station they are not marked trained for, and expiring statutory certifications block scheduling entirely, with the existing notification engine firing 30/15/7-day reminders before a lapse.

A.6 Split-Shift Premium Automation

Lunch-then-dinner split shifts routinely trigger split-shift premium pay under local labor law and are commonly miscalculated by hand. Detecting the split from attendance timestamps and applying the correct premium is one more formula in the payroll component system, not new infrastructure.

A.7 Comp / Void / Walkout ↔ Shift Accountability

An unusual spike in comps, voided items, or a walked table links back to the exact server, shift, and table for manager review — the same audit-ledger traceability principle applied everywhere else in this document, aimed at the events that matter on a restaurant floor.

A.8 Staff Meal as a Tracked Benefit

The free “family meal” before/after a shift is a real, usually untracked, employer cost. One more Financial Ledger component (`staff_meal`, employer cost, non-taxable by default) — consistent with “everything that costs the company money is a ledger entry.”

This specification reflects the full feature set and workflow design discussed and refined across the planning process for Smart Teams EMS Modules 2 and 3, integrated with the existing, production Module 1 attendance system.